



PURBANCHAL UNIVERSITY
HIMALAYAN WHITEHOUSE INTERNATIONAL COLLEGE
PUTALISADAK, KATHMANDU

A
Project-IV
Report
On
“Origin: A Project Gap and Similarity Finder”

By
Sandesh Dotel [Roll No.]
Bhumika Karki [Roll No.]
Diwakar Ray Yadav [Roll No.]

Under the Supervision of

[Supervisor Name]

SUBMITTED TO THE DEPARTMENT OF SCIENCE AND TECHNOLOGY IN
PARTIAL FULFILLMENT OF THE REQUIREMENTS FOR THE DEGREE OF
BACHELOR OF INFORMATION TECHNOLOGY

DEPARTMENT OF SCIENCE AND TECHNOLOGY
Kathmandu, Nepal

September, 2026

CERTIFICATE OF APPROVAL

The Major Project entitled “Origin: A Project Gap and Similarity Finder”, submitted by Sandesh Dotel, Bhumika Karki, and Diwakar Ray Yadav in partial fulfillment of the requirements for the degree of “Bachelor of Information Technology” has been accepted as a bona fide record of work carried out by them in the department.

[Supervisor Name]

Designation: Supervisor

Er. Bimal Sharma

Head of Department

Department of IT, Computer and Electronics

External Examiner

Purbanchal University

ACKNOWLEDGEMENT

We thank our project supervisor, [Supervisor Name], for guiding this project and for the advice that shaped it more than any other: build something you can explain line by line. The review comments before the proposal defense pushed Origin to use its database more actively, and the final system is better for it.

We are grateful to Er. Bimal Sharma, Head of Department, Department of IT, Computer and Electronics, and Er. Aashish Lamsal, Project Coordinator, for the laboratory time, the schedule and the feedback sessions that kept the project on track.

We also thank the faculty members of the Department of IT, Computer and Electronics, Himalayan Whitehouse International College, for their support throughout the semester.

Our classmates in the Bachelor of Information Technology programme discussed their own project ideas with us, and those discussions gave us the idea for Class Radar. Our families gave us the time to finish the work. Thank you.

Sandesh Dotel
Bhumika Karki
Diwakar Ray Yadav

ABSTRACT

Every semester, student groups in project-based programmes propose ideas that repeat earlier projects or clash with a classmate's idea, and the overlap is usually found late. Origin is a web application that checks a project idea at proposal time and suggests what to build instead. Past projects are loaded by copying CSV, TXT, XLSX or DOCX files into a folder that Origin watches. A student types an idea and a map of 41 topics from the BIT syllabus reacts at once: worked ground turns red and untouched topics stay open. A full check scores the idea against the past projects and against the ideas other groups have locked, using three explainable signals (shared tags, shared words and project type) in which rare terms count more, and it tells each match as a story of what is shared, what is new and what to change. A synonym table catches renamed ideas and a claim detector flags buzzwords that the description does not support. The Ideas page turns 42 real problems from Nepal into suggestions, each tied to syllabus topics and shown with its proof, such as how many past projects used the same pair of topics. The MySQL database keeps itself current through triggers, views and stored procedures. The system is built with Java Servlets and Jakarta Server Pages on Apache Tomcat with a MySQL database. All 84 automated tests pass, and the matching formula agrees with human labels on 22 of 25 labelled pairs, as often as the formula it replaced.

Keywords: *project originality, syllabus map, similarity scoring, Java Servlets, MySQL triggers, term weighting*

TABLE OF CONTENTS

CERTIFICATE OF APPROVAL	i
ACKNOWLEDGEMENT	ii
ABSTRACT	iii
TABLE OF CONTENTS	iv
LIST OF FIGURES	vi
LIST OF TABLES	vii
LIST OF ABBREVIATIONS	viii
CHAPTER 1: INTRODUCTION	1
1.1 Background	1
1.2 Problem Statement	1
1.3 Objectives	2
1.3.1 General Objective	2
1.3.2 Specific Objectives	3
1.4 Project Scope	3
1.5 Report Organization	4
CHAPTER 2: LITERATURE OVERVIEW	5
2.1 Background / Theoretical Framework	5
2.2 Study of Related Systems	6
2.3 What's New in Our Project?	7
2.4 Functional and Non-Functional Requirements	8
2.5 Feasibility Study	9
CHAPTER 3: SYSTEM DESIGN AND METHODOLOGY	11
3.1 Software Development Life Cycle (SDLC)	11
3.1.1 SDLC Model Used	11
3.2 Project Timeline - Gantt Chart	12
3.3 System Architecture	12
3.4 Data Flow Diagram (DFD)	14
3.4.1 Level 0 - Context Diagram	14
3.4.2 Level 1 - Detailed DFD	14
3.5 Entity Relationship (ER) Diagram	16
3.6 Use Case Diagram	18
3.7 System Flowchart	19
CHAPTER 4: IMPLEMENTATION	21
4.1 Software and Hardware Requirements	21

4.1.1 Technology Stack and Language Used.....	21
4.2 Module-wise Implementation	22
4.2.1 Authentication and Security Module	22
4.2.2 Similarity Engine Module.....	23
4.2.3 Idea Check and Result Module	25
4.2.4 Class Radar Module.....	28
4.2.5 Gap Map Module	Error! Bookmark not defined.
4.2.6 Syllabus Map and Idea Suggester Module.....	29
4.2.7 DB Folder Import Module	32
4.2.8 Supervisor Desk and Self-Updating Database.....	33
4.3 Testing.....	34
4.3.1 Testing Strategy	34
4.3.2 Test Cases	35
4.3.3 Test Results Summary	38
CHAPTER 5: FINAL OUTCOME	39
5.1 System Output and Analysis.....	39
5.2 Analysis of Output Against Objectives.....	40
5.3 Final System Screenshots.....	41
5.4 Performance and Usability Observations.....	42
CHAPTER 6: CONCLUSION, LIMITATIONS & FUTURE RECOMMENDATIONS	44
6.1 Conclusion	44
6.2 Problems Faced and Limitations.....	44
6.3 Future Recommendations	46
REFERENCES	47
ANNEX.....	48
Annex A: Source Code	48
Annex B: User Manual	49

LIST OF FIGURES

S.No.	Title	Page No.
Figure 3.1	SDLC Model - Iterative and Incremental	11
Figure 3.2	System Architecture Diagram	13
Figure 3.3	Data Flow Diagram - Level 0 (Context Diagram)	14
Figure 3.4	Data Flow Diagram - Level 1	15
Figure 3.5	Entity Relationship (ER) Diagram	17
Figure 3.6	Use Case Diagram	18
Figure 3.7	System Flowchart	20
Figure 4.1	Screenshot - Sign-in Page	23
Figure 4.2	Screenshot - Check Form with Live Corner Preview	25
Figure 4.3	Screenshot - Evaluation Screen with Real Counts	26
Figure 4.4	Screenshot - Renamed Library Idea Caught	27
Figure 4.5	Screenshot - Original Idea Ready to Lock	28
Figure 4.6	Screenshot - Class Radar with a Clash between G06 and G07	29
Figure 4.8	Screenshot - Syllabus Map on Open Ground	30
Figure 4.9	Screenshot - Plot Filter and the Idea Built on It	31
Figure 4.10	Screenshot - Ranked Project Ideas with Proof	32
Figure 4.11	Screenshot - Supervisor Desk	33
Figure 5.1	Screenshot - Landing Page with the Syllabus Map	39
Figure 5.2	Screenshot - Rename Example Computed by the Live Engine	40
Figure 5.3	Screenshot - Check History with CSV Export	41
Figure 5.4	Screenshot - Landing, Result and Ideas Pages on a Phone	42

LIST OF TABLES

S.No.	Title	Page No.
Table 2.1	Study of Related Systems	7
Table 2.2	System Requirements	8
Table 2.3	Feasibility Analysis	10
Table 3.1	Project Gantt Chart	12
Table 4.1	Hardware and Software Requirements	21
Table 4.2	Scoring Signals, Weights and Bands	24
Table 4.3	Database Objects that Keep Origin Current	34
Table 4.4	Test Cases	35
Table 5.1	Measured Response Times	43

LIST OF ABBREVIATIONS

AI	Artificial Intelligence
API	Application Programming Interface
BIT	Bachelor of Information Technology
CSRF	Cross-Site Request Forgery
CSS	Cascading Style Sheets
CSV	Comma-Separated Values
DFD	Data Flow Diagram
DOCX	Office Open XML document (Word)
ER	Entity Relationship
GB	Gigabyte
HMAC	Hash-based Message Authentication Code
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
HTTPS	HyperText Transfer Protocol Secure
IDE	Integrated Development Environment
IEEE	Institute of Electrical and Electronics Engineers
IoT	Internet of Things
JDBC	Java Database Connectivity
JDK	Java Development Kit
JSON	JavaScript Object Notation
JSP	Jakarta Server Pages
JSTL	Jakarta Standard Tag Library
PBKDF2	Password-Based Key Derivation Function 2
RAM	Random Access Memory
SDLC	Software Development Life Cycle
SHA	Secure Hash Algorithm
SQL	Structured Query Language
TC	Test Case
TXT	Plain text file
URL	Uniform Resource Locator
WAR	Web Application Archive
XLSX	Office Open XML spreadsheet (Excel)
XML	Extensible Markup Language

CHAPTER 1: INTRODUCTION

1.1 Background

A Bachelor of Information Technology student builds a software project almost every semester. Each project starts with a proposal, and the proposal starts with an idea. Over the years a college collects hundreds of these projects, and most of them sit in shelves, shared drives and the memory of supervisors. The same kinds of project keep coming back: library systems, attendance systems, hospital systems, online shops and hotel booking sites. In a sample of 71 typical BIT projects that we compiled for this work, 40 are web applications and 15 are in education alone.

Tools that check originality already exist, but they check finished writing. Turnitin compares submitted text with a large collection of documents [1], and code-similarity tools such as MOSS compare finished source code [2]. Neither helps a student at the moment that matters most, which is the week they choose what to build. At that point there is no text to compare, only an idea: a title, a few sentences and a sense of what the system does.

Origin is built for that moment. It compares the structure of an idea with past projects and with this semester's locked ideas, explains every number it shows, and then goes one step further: it shows which topics of the syllabus no past project has built on, and suggests real problems that no group has solved yet, each one made from those topics. Past projects reach the system the easy way: a supervisor copies the college's files into a folder, and Origin reads them by itself.

1.2 Problem Statement

Students choose project ideas without a reliable way to see what has already been built at their college or what their classmates are building now. The result is a steady stream of near-identical proposals, late clashes between groups, and supervisors who have to remember years of past titles to catch them.

Repetition is not the only problem. Two groups in the same class can also pick the same idea in the same week without knowing it, and the clash appears at the proposal defense or, worse, at the mid-term presentation. By then both groups have spent weeks on it.

The Project-IV course gives 10 of its 60 internal marks to title identification and proposal writing, 20 to the mid-term presentation and 30 to the final submission. The idea is chosen first and everything else is built on it: the proposal, the diagrams, the database and the code. A group that learns at the mid-term that its idea was built two years ago, or that another group is building the same thing, has to defend it or start again with half the semester gone.

Three things make the problem hard to solve with existing tools. First, a repeated idea rarely repeats its wording. "Library Management System" becomes "Book Depot Portal", and a word-for-word checker misses it. Second, students pad old ideas with words like "AI" and "prediction" to make them look new, even when nothing in the description supports the claim. Third, telling a student that an idea is taken is only half an answer. Without knowing which areas are still empty, the student simply picks the next crowded idea.

A shared spreadsheet of past titles would help a little, but it cannot tell a student that "Book Depot Portal" and "Library Management System" are the same project, it cannot show a classmate's idea on the day it is chosen, and it cannot say which areas nobody has touched. Those things need software that looks at what an idea does, not only at what it is called.

At the same time, the problems around the students stay unsolved. Households in Kathmandu wait for private water tankers without knowing their place in the queue, farmers sell vegetables without knowing that day's wholesale price at Kalimati, and rural health posts run out of essential medicines before the district store notices. A three-person team can build a working system for any of these in one semester, yet our sample of past projects has only 2 projects each in agriculture, in environment and disaster, and in governance.

The people affected are students, who lose time; supervisors, who carry the whole checking load; and the department, which receives the same few projects year after year while real local problems go unsolved.

1.3 Objectives

1.3.1 General Objective

To design and develop Origin, a web application that loads past projects from files dropped into a folder, checks a student project idea against them and against this semester's

locked ideas at proposal time, shows which syllabus topics are untouched, and suggests real problems worth building, with a database that keeps its own records current.

1.3.2 Specific Objectives

- To analyse how BIT project ideas repeat and to study existing originality and plagiarism tools.
- To design a layered web architecture and a normalised MySQL schema whose triggers, views and stored procedures keep the data current without manual upkeep.
- To develop an explainable similarity engine that compares an idea with past projects and locked class ideas and resists renaming and buzzword padding.
- To develop a Syllabus Map and an idea suggester that turn real problems and syllabus topics into ranked project suggestions, each with its proof.
- To implement Class Radar locking, supervisor review, semester archiving and a DB folder that loads past projects from CSV, TXT, XLSX and DOCX files by itself.
- To test the system for correctness, security and performance.

Objectives 1 and 2 are covered in Chapters 2 and 3, objectives 3 to 5 in Chapter 4, and objective 6 in Chapters 4 and 5.

1.4 Project Scope

Origin is built for a single college running BIT Project-IV. It runs on a college server or a lab machine, and students and supervisors use it from a browser. The scope was set early and kept.

Inclusions:

- Idea check: explainable scoring against the past-project corpus and this semester's locked ideas, with synonym folding, a claim detector and a result page that shows every component of the score.
- Class Radar: one locked idea per group per semester, automatic clash detection between groups, and supervisor review.
- Syllabus Map and ideas: a live map of 41 syllabus topics shaded by how many past projects used each, and ranked suggestions built from a bank of real problems and the syllabus topics they use.

- Self-updating database: triggers for counters and the audit trail, a stored procedure that archives each semester into the corpus, a folder watcher that loads past projects from CSV, TXT, XLSX and DOCX files, and a background thread that refreshes suggestions.
- User roles: Student and Supervisor.

Exclusions:

- Checking finished reports or source code for copied text, which is the job of tools such as Turnitin.
- A mobile application, email notifications and multi-college use.
- Real-time chat between groups.
- Reading PDF files. They must be converted to DOCX or CSV first, because the JDK cannot read them and the project avoids libraries outside the syllabus.

Origin assumes that students describe their idea honestly with meaningful tags. A student who writes a genuinely new description for an old idea can pass the check. We accept that limit, because at that point the student has done the thinking the check is meant to encourage.

1.5 Report Organization

This report has six chapters. Chapter 1 introduces the problem, the objectives and the scope. Chapter 2 reviews the theory behind the system and the tools that already exist, and sets out the requirements and the feasibility study. Chapter 3 describes the development model, the timeline and the design: architecture, data flow, database, use cases and the main flowchart. Chapter 4 explains how each module was built and how the system was tested. Chapter 5 compares the finished system with each objective, and Chapter 6 closes with the problems we met, the limits of the system and the work that could follow. The references and the annexes, including the user manual, come after the last chapter.

CHAPTER 2: LITERATURE OVERVIEW

2.1 Background / Theoretical Framework

Origin rests on a small number of well-known ideas, chosen because each one can be explained with a pencil and a piece of paper. This section describes them in the order the system uses them.

Set similarity. Origin treats an idea as sets: a set of tags and a set of meaningful words. Two sets are compared with the Jaccard index, the size of their intersection divided by the size of their union [3]. Two ideas with the tags {library, booking, notification} and {library, student, notification} share two tags out of four distinct ones, so their tag similarity is 0.5. The measure is easy to compute, easy to check by hand, and does not reward an idea for simply having more tags.

Term weighting. A word that only a few past projects used says more about an idea than a word that most of them used. Origin weighs each term by $1 / (1 + \ln(1 + n))$, where n is the number of past projects that use it, the same idea as inverse document frequency in information retrieval [4]. A term nobody used weighs 1.0 and a term used by nine projects weighs about 0.3. Overlap is then measured in two ways and combined 80% and 20%: the weighted Jaccard index, and the share of the idea's own weight that the other project already has. The second part stops a small idea that sits inside a large old project from being scored as half similar.

Text normalisation. Before words are compared they are lower-cased, split on anything that is not a letter or digit, and filtered through a stop-word list, a standard step in information retrieval [4]. Origin's list also removes words that describe nothing about the project, such as "system", "management", "online" and "portal". Simple plural folding turns "libraries" into "library". A synonym table then maps words like "book", "lending" and "catalog" to the canonical tag library, which is what makes renaming ineffective.

Relational design. The data model follows the relational model and is normalised to third normal form, so each fact is stored once [5]. Many-to-many links, such as projects and their tags, use junction tables with composite primary keys. Rules that must always hold, such as one locked idea per group per semester, are enforced by the database with unique keys and check constraints rather than by application code alone.

Active databases. A trigger is a stored program that runs automatically when a row is inserted, updated or deleted, and a stored procedure is a named program that runs a sequence of statements on request [6]. Origin uses both so that the database maintains its own counters and audit trail and can move a whole semester into the corpus inside one transaction.

Server-side Java. A servlet is a Java class that receives an HTTP request and produces a response inside a container, and Jakarta Server Pages turn templates into HTML on the server [7]. Apache Tomcat is the container used to run them [8]. Origin follows a three-layer pattern: servlets handle HTTP, services hold the logic, and data access objects talk to MySQL through JDBC.

Password storage. Passwords are never stored. Origin stores a PBKDF2 hash, which runs HMAC-SHA256 many thousands of times over the password and a random salt [9]. The salt makes identical passwords produce different hashes, and the repetition makes guessing slow, which is the approach current guidance recommends [10].

Syllabus topics. Origin describes each topic of the fourth-semester syllabus, such as page replacement, triggers or the t-test, with a short list of keywords. A past project is linked to every topic whose keywords appear in its title, description or tech stack. Counting the projects per topic, and per pair of topics, gives the Syllabus Map and the proof shown beside each suggestion. The method is plain keyword matching on purpose: anyone can check a count by hand, and a wrong count points straight at the keyword that caused it.

2.2 Study of Related Systems

We reviewed seven systems that students and supervisors use today, or could use, to judge whether a project is original. Table 2.1 summarises them.

Table 2.1: Study of Related Systems

S.N.	Title / Year	Technology	Key Features	Limitations
1	Turnitin, Turnitin LLC, 1998 to present [1]	Web service, text matching	Similarity report for submitted writing, large document database	Works on finished text; cannot judge an idea before it is written; paid licence
2	iThenticate, Turnitin LLC [12]	Web service, text matching	Plagiarism screening for research manuscripts	Same after-the-fact model; built for publishers, not project proposals
3	MOSS, Schleimer et al., 2003 [2]	Server, winnowing fingerprints	Finds similar source code across student submissions	Needs finished code; says nothing about the idea itself
4	JPlag, Prechelt et al., 2002 [13]	Java, token string matching	Detects copied programs even after renaming variables	Code only; no view of past projects or current classmates
5	DSpace institutional repository [14]	Java web application, keyword search	Stores and lists past theses and project reports	Passive archive; gives no similarity score
6	Google Scholar [15]	Web search engine	Finds published research on a topic	Covers research papers, not a college's own student projects
7	Manual supervisor review	Human memory	Experienced supervisors recognise common repeats	Does not scale, misses renamed ideas and same-week clashes between groups

Note: The related systems listed above are cited in IEEE format in the References section.

The pattern is clear. Every automated tool in Table 2.1 compares finished work: written text or source code. None of them looks at an idea before development starts, none knows what other groups in the same class are doing this week, and none tells a student where the unexplored areas are. The repository and the search engine hold useful information, but a student has to search them by hand and judge the similarity alone.

2.3 What's New in Our Project?

Origin fills the gaps found in Section 2.2:

- It works at proposal time, on the idea itself, before any report or code exists.
- It compares an idea with two pools at once: the college's own past projects and the ideas other groups locked this semester, so same-week clashes appear on the Class Radar for both groups.

- It explains every score as tags in common, words in common and project type, lets rare terms count more than common ones, tells each match as what is shared, what is new and what to change, and resists renaming through a synonym table and buzzword padding through a claim detector.
- It answers the next question as well: a Syllabus Map shows which topics of the syllabus no past project has used, and the Ideas page offers real problems from Nepal built from those topics, each with proof such as how many past projects used the same pair of topics.
- Its database keeps itself current. Triggers maintain counters and the audit trail, a stored procedure moves approved ideas into the corpus at the end of each semester, and a background thread refreshes the suggestions.
- Past projects load themselves: files dropped into a folder are read within about ten seconds, rows already in the corpus are skipped, and the map and the suggestions update without anyone clicking.

2.4 Functional and Non-Functional Requirements

The requirements below come from the problem statement and from the feedback we received at the proposal defense (see Table 2.2).

Table 2.2: System Requirements

Requirement Type	Description	Priority
Functional - User Auth	Students register with a group code; students and supervisors sign in and out securely.	High
Functional - Idea check	The system scores an idea against past projects and locked class ideas and shows each component of the score.	High
Functional - Anti-gaming	Renamed ideas are caught through synonyms; unsupported buzzwords are flagged.	High
Functional - Class Radar	Each group locks one idea per semester; clashes between groups are recorded and shown.	High
Functional - Syllabus Map and ideas	A live map of syllabus topics reacts to a typed idea; the Ideas page ranks suggestions with proof and puts plain CRUD ideas last.	High
Functional - Past-project import	Files dropped into the DB folder (CSV, TXT, XLSX, DOCX) are read within about ten seconds; bad rows are skipped and reported.	High

Requirement Type	Description	Priority
Functional - Supervisor	Supervisors approve or send back ideas, correct matches, close the semester and download a report.	Medium
Functional - History and export	Every check is stored; students download their history as CSV.	Medium
Non-Functional - Performance	A check completes and a page loads within 2 seconds under normal load.	High
Non-Functional - Security	Passwords are hashed with a salt; forms carry CSRF tokens; roles are enforced on every request.	High
Non-Functional - Explainability	Every score can be recomputed by hand from the tags, words and type shown on screen.	High
Non-Functional - Usability	The interface works on desktop and phone screens without training.	Medium
Non-Functional - Reliability	A bad or half-copied file never stops the system and never half-fills the corpus.	Medium

2.5 Feasibility Study

The feasibility study evaluates whether the proposed project can be successfully developed and deployed given the available resources, technology, and timeline.

Technically, every part of Origin uses topics from the fourth-semester syllabus: Java, JDBC, Servlets and JSP from Programming in Java, and SQL, triggers, procedures, normalisation and transactions from Database Management System. Operationally, students already write a title and a short description for every proposal, so Origin asks for nothing new. Economically, all tools are free. Table 2.3 summarises the analysis.

Table 2.3: Feasibility Analysis

Feasibility Type	Analysis	Verdict
Technical	Java 17, Apache Tomcat 10.1, MySQL 8 and JDBC are open-source, well documented and taught in the syllabus.	Feasible
Operational	Students enter the same information they already write in a proposal; the pages need no training.	Feasible
Economic	All tools are free, and no paid service is needed.	Feasible
Schedule	The work fits into five increments over one semester, as shown in the Gantt chart (Table 3.1).	Feasible
Legal	Only project metadata and student-entered text are stored; libraries and fonts are open-source or free licences.	Feasible

Based on the above analysis, the project is determined to be fully feasible within the given academic context, available tools, and team capability.

CHAPTER 3: SYSTEM DESIGN AND METHODOLOGY

3.1 Software Development Life Cycle (SDLC)

3.1.1 SDLC Model Used

We followed the iterative and incremental model [16]. The system was split into five increments, and each one went through planning, design, building and testing before the next began (Figure 3.1). The first increment delivered the schema, the seed data and login. The second added the rule engine and the check flow, which made Origin usable on its own. The third added Class Radar, the fourth the idea generator and the DB-folder import, and the fifth the Syllabus Map, the suggester, the full test suite and this report.

The model suited a three-person team with a heavy exam load. After the second increment we always had a working system to demonstrate, so later increments could only improve the demo, never break it. It also absorbed the changes requested at the proposal defense: the call for more active use of the database became the triggers, the archive procedure and the self-refreshing suggestions of the third and fourth increments.

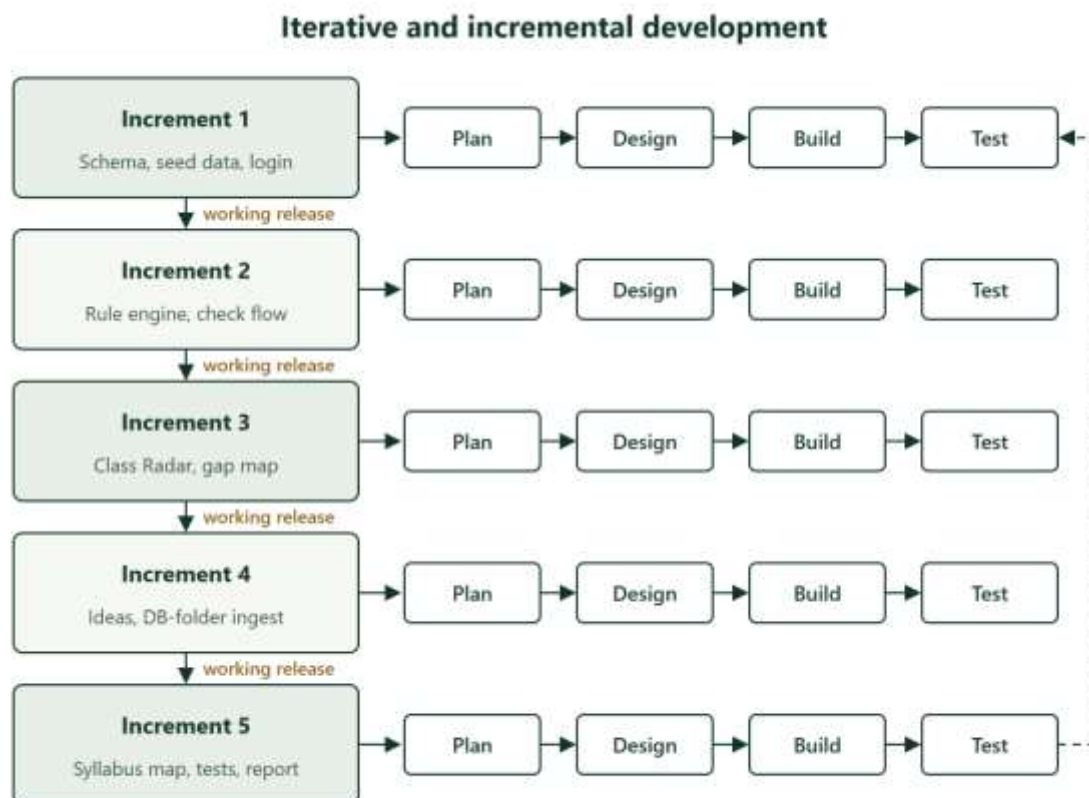


Figure 3.1: SDLC Model - Iterative and Incremental

3.2 Project Timeline - Gantt Chart

The Gantt chart in Table 3.1 shows the timeline from February to October 2026. Shaded cells mark the months in which each activity ran.

Table 3.1: Project Gantt Chart

Activity	Feb	Mar	Apr	May	Jun	Jul	Aug	Sep	Oct
Requirement study and literature review									
Proposal writing and proposal defense									
System and database design									
Increments 1 and 2: schema, seed data, rule engine									
Increment 3: Class Radar									
Increment 4: idea generator and DB-folder import									
Increment 5: Syllabus Map, suggester, supervisor desk									
Testing and fixes									
Report and final presentation									

Note: Shaded cells show the planned duration of each activity.

3.3 System Architecture

Origin uses a layered client-server architecture (Figure 3.2). The browser receives HTML rendered from JSP pages. Every request first passes through a security filter, which checks the session, the role and the CSRF token and adds the security headers. Servlets act as controllers: they read the request, call a service and forward the result to a JSP view. One small servlet, /peek, answers the live preview with JSON while a visitor types.

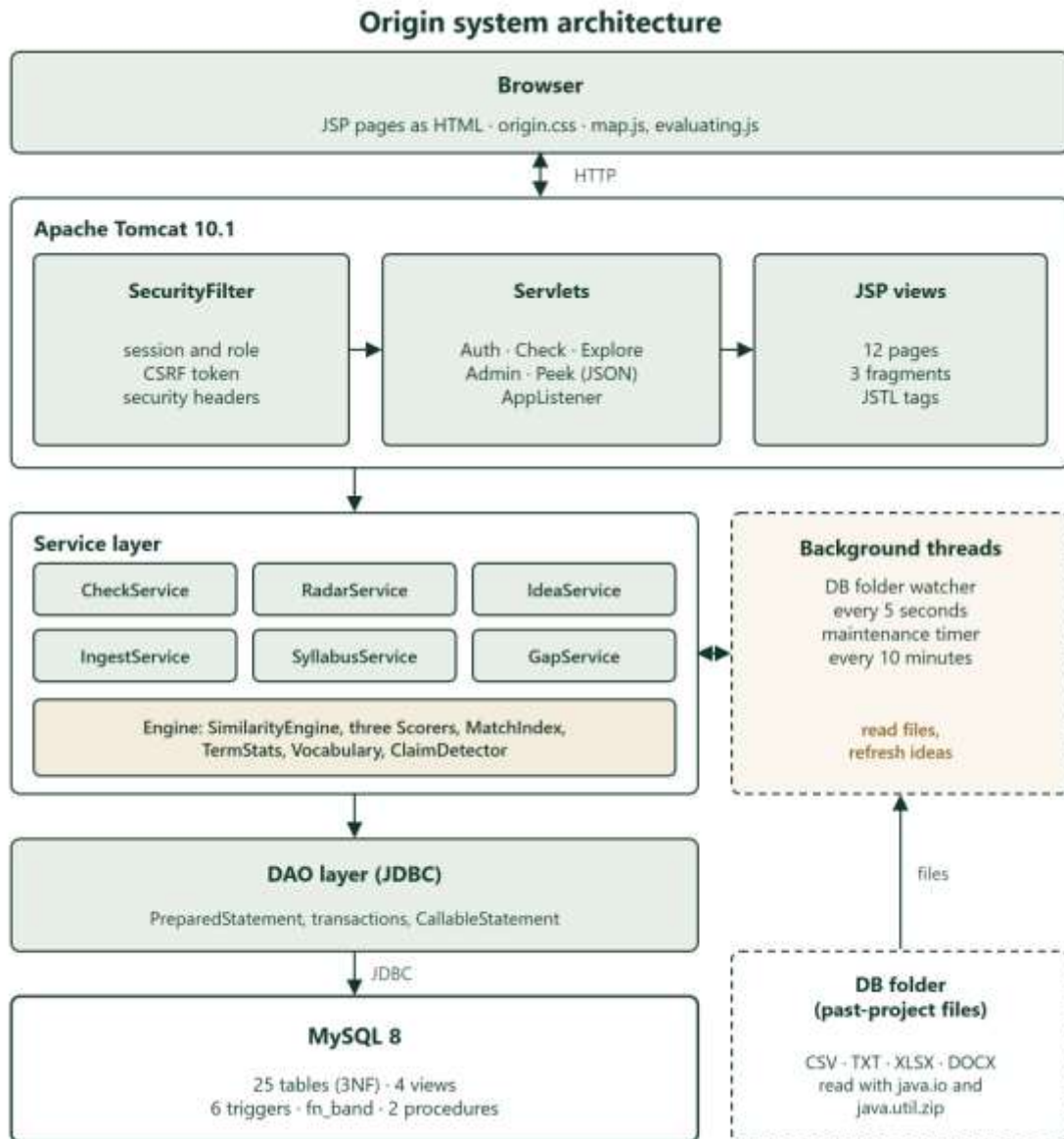


Figure 3.2: System Architecture Diagram

The service layer holds the logic. CheckService validates and scores ideas, RadarService handles locking and clashes, IdeaService builds the suggestions, IngestService reads the CSV files from the DB folder, SyllabusService links projects to topics and AuthService signs people in. The similarity engine sits beside them as plain Java classes with no database access, which is what makes it easy to test. The data access layer uses JDBC with PreparedStatement for every query, transactions for every multi-step write, and CallableStatement for the stored procedures.

Three background jobs run beside the request threads. A watcher thread looks in the DB folder every five seconds and hands each new or changed file to IngestService. A timer thread refreshes the idea suggestions every ten minutes, and a small worker pool

refreshes them after every lock and every import. The folder is outside the system: when it is empty, the watcher simply does nothing.

3.4 Data Flow Diagram (DFD)

3.4.1 Level 0 - Context Diagram

At the highest level, Origin exchanges data with two people and one external source (Figure 3.3). Students send ideas, tags and lock requests and receive verdicts, matches, the syllabus map, suggestions and the radar. Supervisors send review decisions, corrections and the command to close a semester, and receive the locked ideas, clashes, the ingest log and the semester report. The DB folder supplies past-project files.

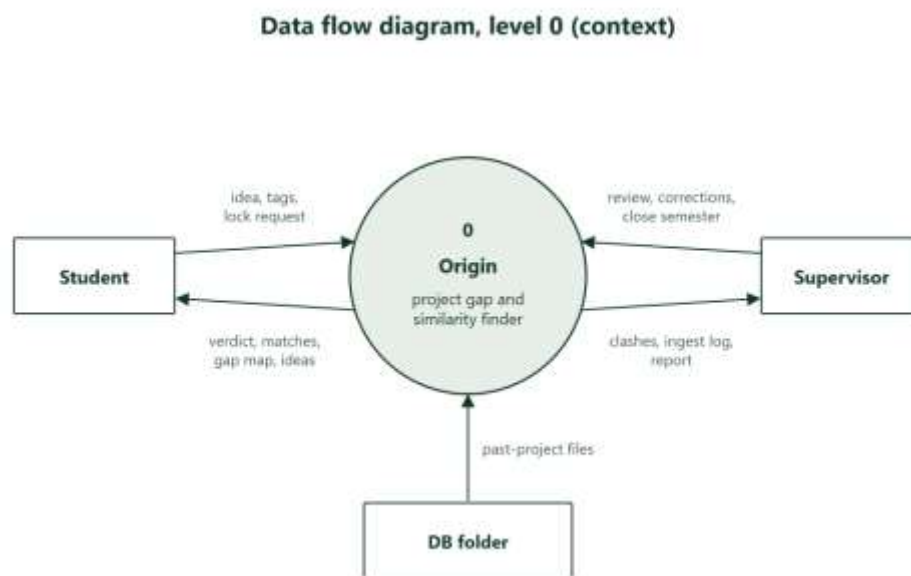


Figure 3.3: Data Flow Diagram - Level 0 (Context Diagram)

3.4.2 Level 1 - Detailed DFD

Figure 3.4 breaks the system into six processes and seven data stores. Authentication (1.0) reads the users. Checking an idea (2.0) reads the corpus and the locked proposals and writes the check with its matches. Locking (3.0) writes the proposal and its clashes, and the audit log is written by triggers. Building the map and ideas (4.0) reads the corpus, its topics and the proposals and writes the suggestions. Review and archiving (5.0) updates proposals and, at the end of the semester, writes approved ideas into the corpus. Reading the DB folder (6.0) takes files from the folder, adds new projects to the corpus with their topics, and writes one line per file to the ingest log.

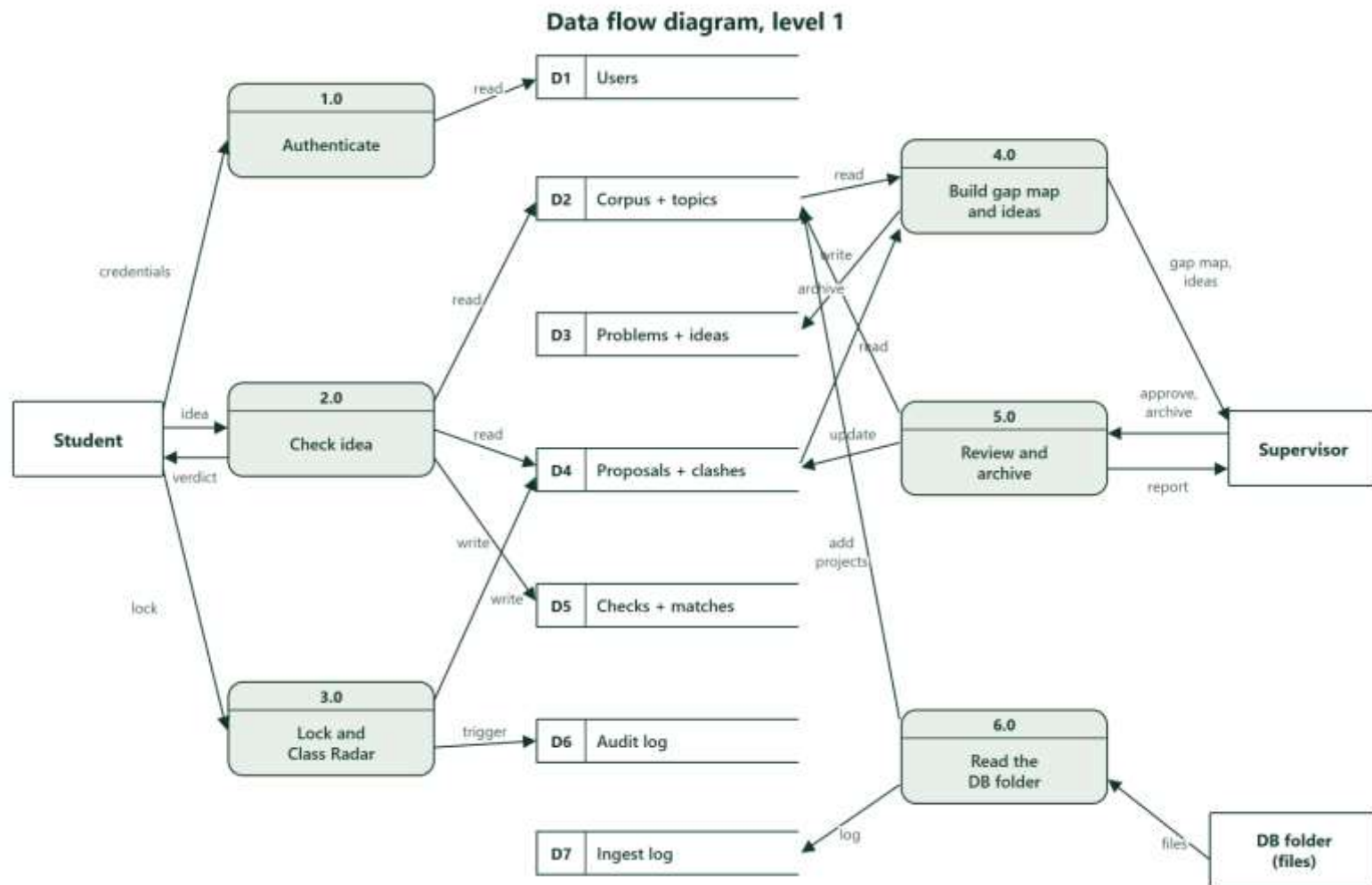


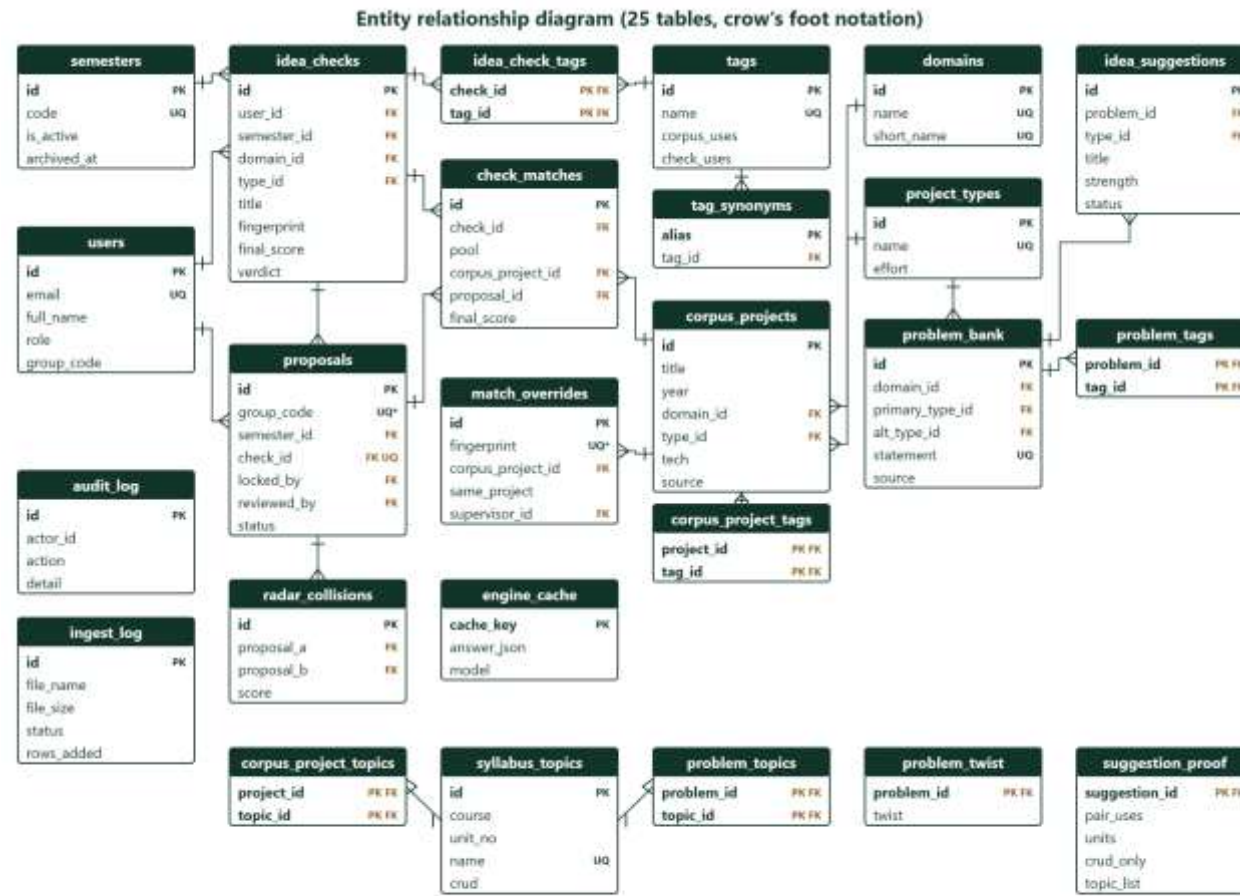
Figure 3.4: Data Flow Diagram - Level 1

3.5 Entity Relationship (ER) Diagram

The database has 25 tables in third normal form (Figure 3.5). The historical corpus lives in `corpus_projects`, linked to tags through `corpus_project_tags` and to syllabus topics through `corpus_project_topics`. Every check a student runs is stored in `idea_checks` with its tags in `idea_check_tags` and its scored matches in `check_matches`. A match points either to a corpus project or to a proposal, never both, which a check constraint enforces. When a group locks a check it becomes a row in `proposals`, whose composite unique key on `group_code` and `semester_id` enforces one idea per group per semester. Clashes between proposals are rows in `radar_collisions`.

The ideas side has its own tables: `problem_bank` holds real problems with their tags in `problem_tags` and their syllabus topics in `problem_topics`, `problem_twist` holds one sentence per problem, and `idea_suggestions` holds one row per problem and suitable project type, with `suggestion_proof` storing the numbers shown as its proof. Supervisor corrections live in `match_overrides` and the `audit_log` is written only by triggers. `engine_cache` belongs to an optional integration that is switched off in this version. Domains, project types and semesters are small reference tables.

Version 3 added six tables: `ingest_log` (one row per file version read from the DB folder), `syllabus_topics` (the 41 topics, each with its course, unit and keywords), `corpus_project_topics`, `problem_topics`, `problem_twist` and `suggestion_proof`. The table `corpus_projects` also gained a `tech` column, because the tech stack is where syllabus topics show. An upgrade step adds the column to a database made by an earlier version, so no data is lost.



Not drawn, to keep the lines readable: domains and project_types are also referenced by idea_checks, problem_bank and idea_suggestions;
 tags by problem_tags; users by match_overrides. UQ* = part of a composite unique key: (group_code, semester_id) and (fingerprint, corpus_project_id).
 Bottom row: corpus_project_topics.project_id refers to corpus_projects; problem_topics and problem_twist refer to problem_bank; suggestion_proof refers to idea_suggestions.

Figure 3.5: Entity Relationship (ER) Diagram

3.6 Use Case Diagram

As shown in Figure 3.6, the system supports thirteen primary use cases across two actor roles, plus two use cases started by the system itself. A student registers, tries an idea on the Syllabus Map, checks an idea, reads the result, browses ideas, locks an idea for the group, views the Class Radar and downloads the check history. A supervisor can do everything a student can and also reviews locked ideas, corrects matches, closes the semester, downloads the semester report and drops past-project files into the DB folder. The folder watcher and the timer drive the reading of files and the refresh of suggestions.

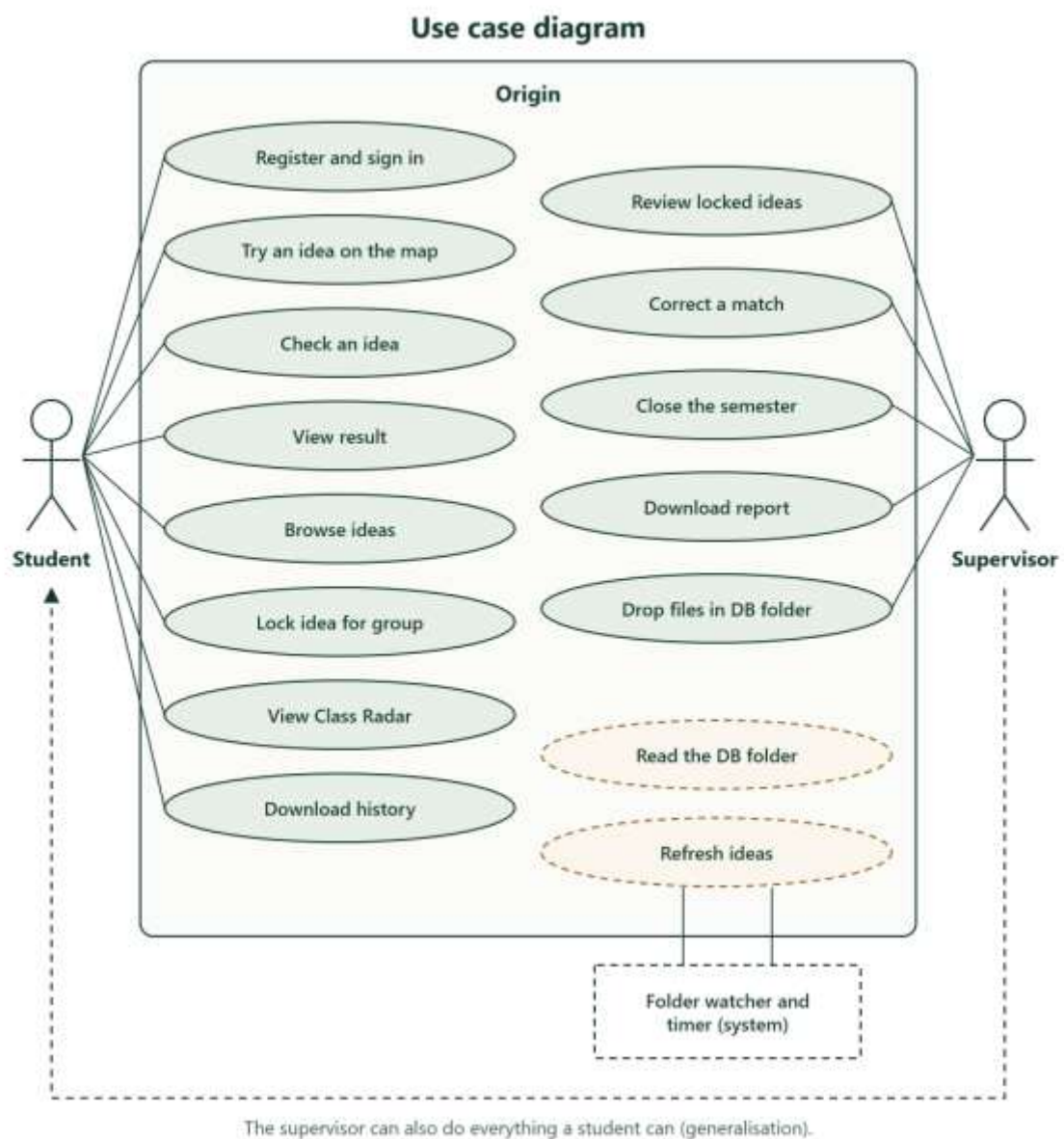


Figure 3.6: Use Case Diagram

3.7 System Flowchart

Figure 3.7 follows one idea from sign-in to lock. Invalid input loops back with field errors. A valid idea is normalised, scored against both pools, adjusted by any remembered supervisor correction, banded and saved in one transaction. An evaluation screen then shows what Origin looked at for about two and a half seconds, and the result page opens. Locking is refused for a duplicate or for a group that has already locked an idea; otherwise the proposal is saved, compared with every other group's idea, and any clash is recorded.

System flowchart: checking and locking an idea

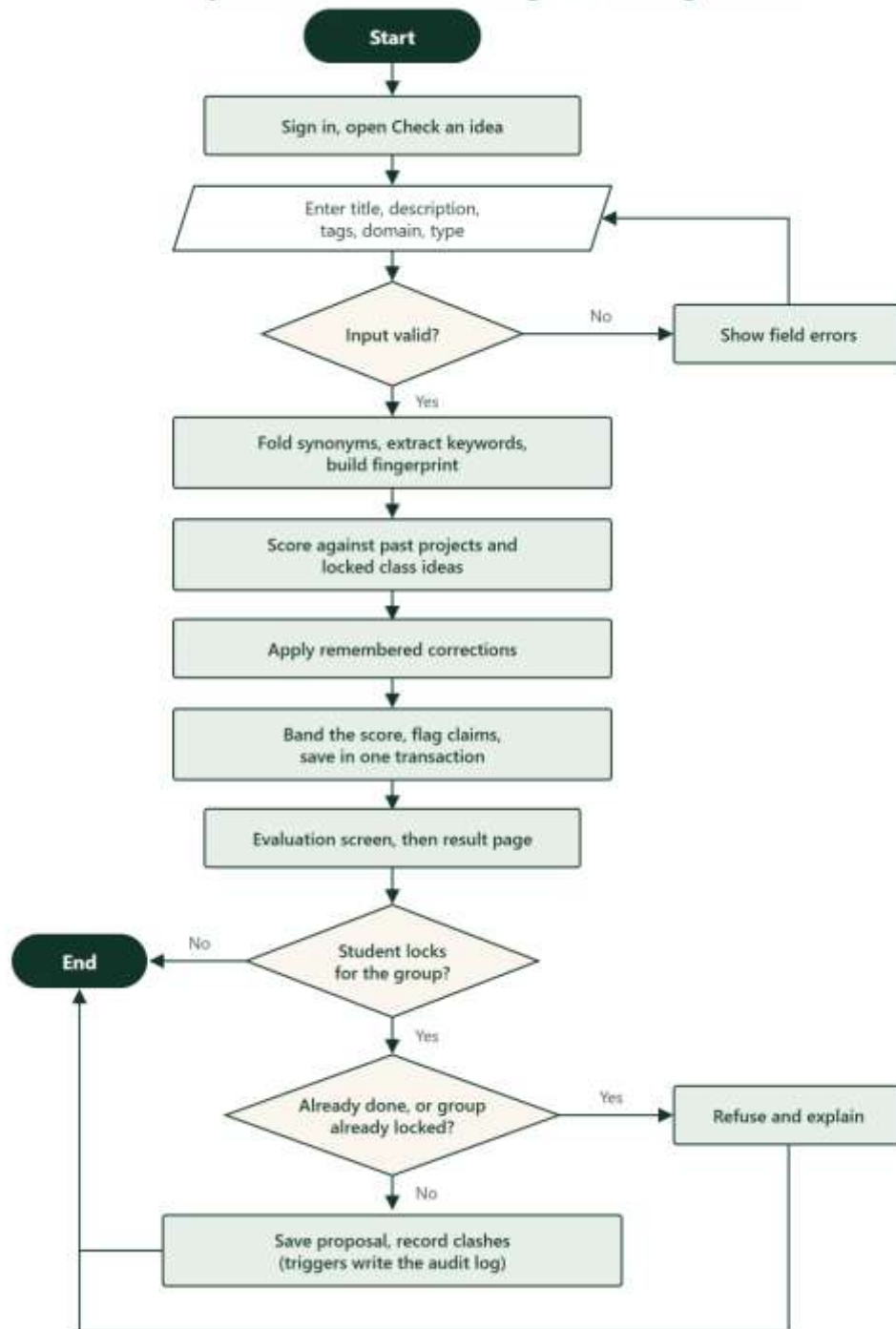


Figure 3.7: System Flowchart

CHAPTER 4: IMPLEMENTATION

4.1 Software and Hardware Requirements

Table 4.1 lists the hardware and software used to develop and run Origin, with versions.

Table 4.1: Hardware and Software Requirements

Component	Specification / Tool	Purpose
Processor	Intel Core i3 or equivalent (1 GHz+)	Compiling and running the server
RAM	Minimum 4 GB (8 GB recommended)	Tomcat, MySQL and the IDE together
Storage	Minimum 2 GB free disk space	Source, dependencies and database files
Operating System	Windows 10 / 11 or Ubuntu 20.04 or later	Development and deployment
Browser	Google Chrome / Mozilla Firefox (latest)	Using and testing the interface
Code Editor / IDE	IntelliJ IDEA / Visual Studio Code	Development environment
Language	Java 17 (JDK 17 or newer)	All server-side code
Web container	Apache Tomcat 10.1 (Servlet 6.0, JSP 3.1)	Runs servlets and JSP pages
View layer	JSP with JSTL 3.0, hand-written CSS and JavaScript	Server-rendered pages
Database	MySQL 8.4 with Connector/J 8.4	Data, triggers, views and procedures
JSON	Jakarta JSON Processing (Eclipse Parsson 1.1)	The live preview reply and the idea data on the home page
Build tool	Apache Maven 3.9	Dependencies, compiling, tests and WAR packaging
Testing	JUnit 5, a Chrome DevTools driver in Node, Playwright	Automated tests, browser checks and diagrams
Version Control	Git & GitHub	Source code management

4.1.1 Technology Stack and Language Used

Every choice below comes from the fourth-semester syllabus unless stated otherwise.

- **Back-end: Java Servlets on Apache Tomcat 10.1.** Servlets are the syllabus way to write web applications in Java, and Tomcat is the container named in it. Nineteen servlets act as controllers, one filter handles security and a context listener starts the application [7], [8], [17].
- **Front-end: JSP with JSTL.** Pages are rendered on the server, so they work without JavaScript. Two small scripts add the live Syllabus Map and the evaluation screen on top.
- **Database: MySQL 8 through JDBC.** A relational database suits data made of projects, tags and the links between them. JDBC with PreparedStatement protects every query from SQL injection, and transactions keep multi-step writes all-or-nothing [5], [6], [18].
- **File reading: java.io and java.util.zip.** CSV and TXT files are read with BufferedReader. XLSX and DOCX files are zip archives of XML, so they are opened with the zip classes and the XML parser that come with the JDK, and no extra library is needed.
- **Version control: Git & GitHub** for collaborative development and code versioning.
- **Deployment: localhost or any Tomcat 10.1 server.** A launcher starts an embedded Tomcat with one command for development and the demonstration; the same code builds a WAR file for a standalone Tomcat.

4.2 Module-wise Implementation

Origin has eight modules. Each description below points to the classes that implement it and to the screenshots taken from the running system.

4.2.1 Authentication and Security Module

AuthService registers students and signs people in. A password is hashed with PBKDF2 using HMAC-SHA256, 120,000 iterations and a random 16-byte salt, and the hash is compared in constant time so that the response time does not leak how much of it matched [9], [10]. A failed login for an unknown email costs the same hashing time as a wrong password, so timing cannot reveal which emails have accounts. After five failed attempts from one address in five minutes, further attempts receive HTTP 429.

SecurityFilter runs before every request. It sends anonymous users to the sign-in page, returns 403 when a student opens a supervisor page, rejects any POST without the session's CSRF token, and sets the Content-Security-Policy, X-Frame-Options and X-Content-Type-Options headers. Session cookies are HttpOnly with SameSite=Lax, and a new session id is issued at login. Figure 4.1 shows the sign-in page.

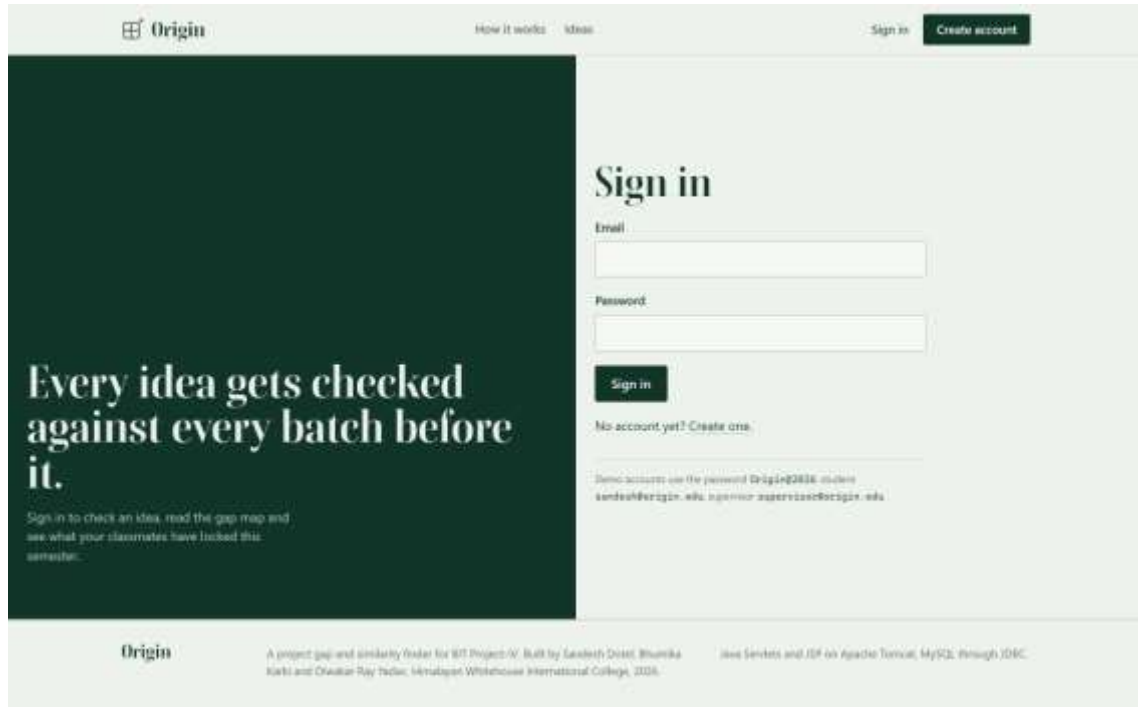


Figure 4.1: Screenshot - Sign-in Page

4.2.2 Similarity Engine Module

The engine is plain Java with no database access. An idea becomes a Candidate: a set of canonical tags, a set of meaningful keywords and a project type. Three scorers implement one Scorer interface, and SimilarityEngine loops over a List<Scorer> without knowing which concrete class it is calling. This is where inheritance and polymorphism do real work in the project. The scorers compare two sets with Overlap, which counts each term by the weight kept in TermStats.

```
public interface Scorer {
    String name();
    double weight();           // all weights add up to 1.0
    double score(Candidate idea, Candidate other); // 0 to 1
}

public class TagScorer implements Scorer {
    private final TermStats stats;
    public double weight() { return 0.45; }
    public double score(Candidate idea, Candidate other) {
        return Overlap.of(idea.tags(), other.tags(), stats);
    }
}
```

}

The weights and bands are fixed and published, as Table 4.2 shows. Tags weigh most because they describe what a project does, and function does not change when the title does. Each term counts by its weight, $1 / (1 + \ln(1 + n))$, where n is the number of past projects that use it, so a word used by one project says more than a word used by sixty. Overlap is 80% the weighted Jaccard index and 20% the share of the idea's own weight that the other project already has. The type counts only when the tags or words already overlap, so two unrelated web applications do not start at 20%. The same thresholds are written once in Java and once in the SQL function `fn_band`, and a test checks that they agree.

Table 4.2: Scoring Signals, Weights and Bands

Item	Rule	Value
Term weight	$1 / (1 + \ln(1 + n))$, n = past projects that use the term	Rare terms count more
Overlap	80% weighted Jaccard index + 20% share of the idea's weight already present	Used for tags and words
Tags in common	Overlap of canonical tag sets	45% of the score
Words in common	Overlap of title and description keywords	35% of the score
Same type	1 if both are the same project type; counted only when tags or words overlap	20% of the score
Looks original	Final score below 0.35	Band ORIGINAL
Overlaps	Final score from 0.35 to 0.59	Band SIMILAR
Already done	Final score of 0.60 or more	Band DUPLICATE
Not enough data	Fewer than 15 projects in the corpus	Band LOW_CONFIDENCE

Two guards sit beside the scorers. The Vocabulary class folds 87 synonyms into 104 canonical tags, so "books, lending, reservation" becomes {library, booking}. The ClaimDetector watches six kinds of claim, including AI, prediction, IoT and blockchain. If one appears in the title or tags but the description names no data, device or method to back it up, the result page says so.

Three more parts make the result easy to read and quick to compute. Coverage measures how much of the idea's weight the three closest projects hold together, which shows a mash-up of two old projects. Story splits each match into the terms shared, the

terms only the idea has and the terms only the old project has, with tags first, and writes one sentence on what to change. MatchIndex keeps, for each term, the list of projects that contain it, like the index at the back of a book, so a check scores only the projects that share at least one term. A project that shares no term scores zero anyway, so the answer is the same as scoring all of them, and a test compares the two.

4.2.3 Idea Check and Result Module

The check form asks for a title, a domain, a project type, a description, an optional problem statement and tags. While the student picks a domain and type, a panel beside the form shows how crowded that corner of the map already is (Figure 4.2).

Check an idea

Describe what the project does, not how impressive it sounds. Origin compares it with every past project and with the ideas your classmates have linked.

Title
Book Depot Portal

Domain
Education

Project type
Web App

What it does
Two to five sentences. What you see it, what they do with it, and how it works.
Students reserve books online from the college collection and get a reminder before the return date. Librarians see who has which book.

The problem it solves (optional)
What is here today and how. Good problem statements help the next batch find their too.

Tags
What the project does, separated by commas. Pick from the list or type your own.
books, lending, reservation, notification

Check originality

Your corner of the map
9 projects here
7 from past batches and 2 locked this semester in Education / Web App.

How the score works
Three plain signals, added up with fixed weights. You can check every number by hand.

Signal	Percentage of the score
Tags in common	45%
Words in common	35%
Same type	20%

Below 35% looks original. 35 to 59% overlaps. 60% and above is already there.

Figure 4.2: Screenshot - Check Form with Live Corner Preview

CheckService validates the input, builds the Candidate, ranks the five closest past projects and the three closest locked class ideas, applies any remembered supervisor correction, and saves the check, its tags and its matches in one transaction. The student then sees an evaluation screen for about two and a half seconds (Figure 4.3). It lists the stages Origin went through with their real counts, for example 32 terms found, 71 projects

searched of which 38 share a term, and 8 locked ideas compared. A link skips ahead, and a visitor who prefers reduced motion goes straight to the result.

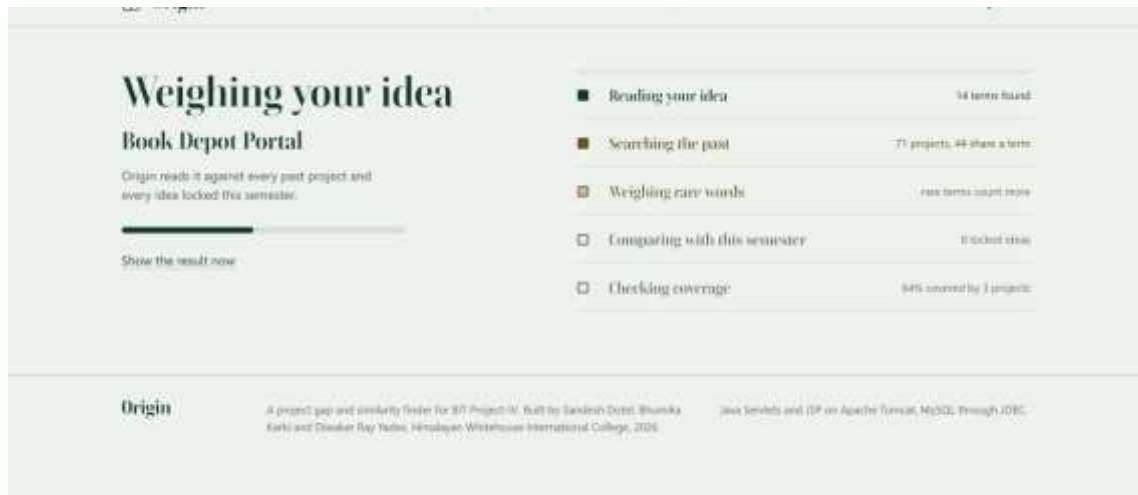


Figure 4.3: Screenshot - Evaluation Screen with Real Counts

Figure 4.4 shows the rename test from Chapter 1. "Book Depot Portal", described as students reserving books and getting reminders, scores 62% and lands in the Already done band. Its closest past project is the Online Library Book Lending Portal from 2021: 58% of tags and 44% of words in common, and the same type. The page tells the match as a story: booking, library and student are shared; notification and depot belong only to this idea; search and receive belong only to the old project. One sentence sums it up: the real difference is notification and depot, and that should be the core of the project. Group G01's Library Study Seat Booking shares half the tags but only 11% of the words, so it stays at 46%: reserving a seat is not reserving a book. The three closest projects together cover 64% of the idea.

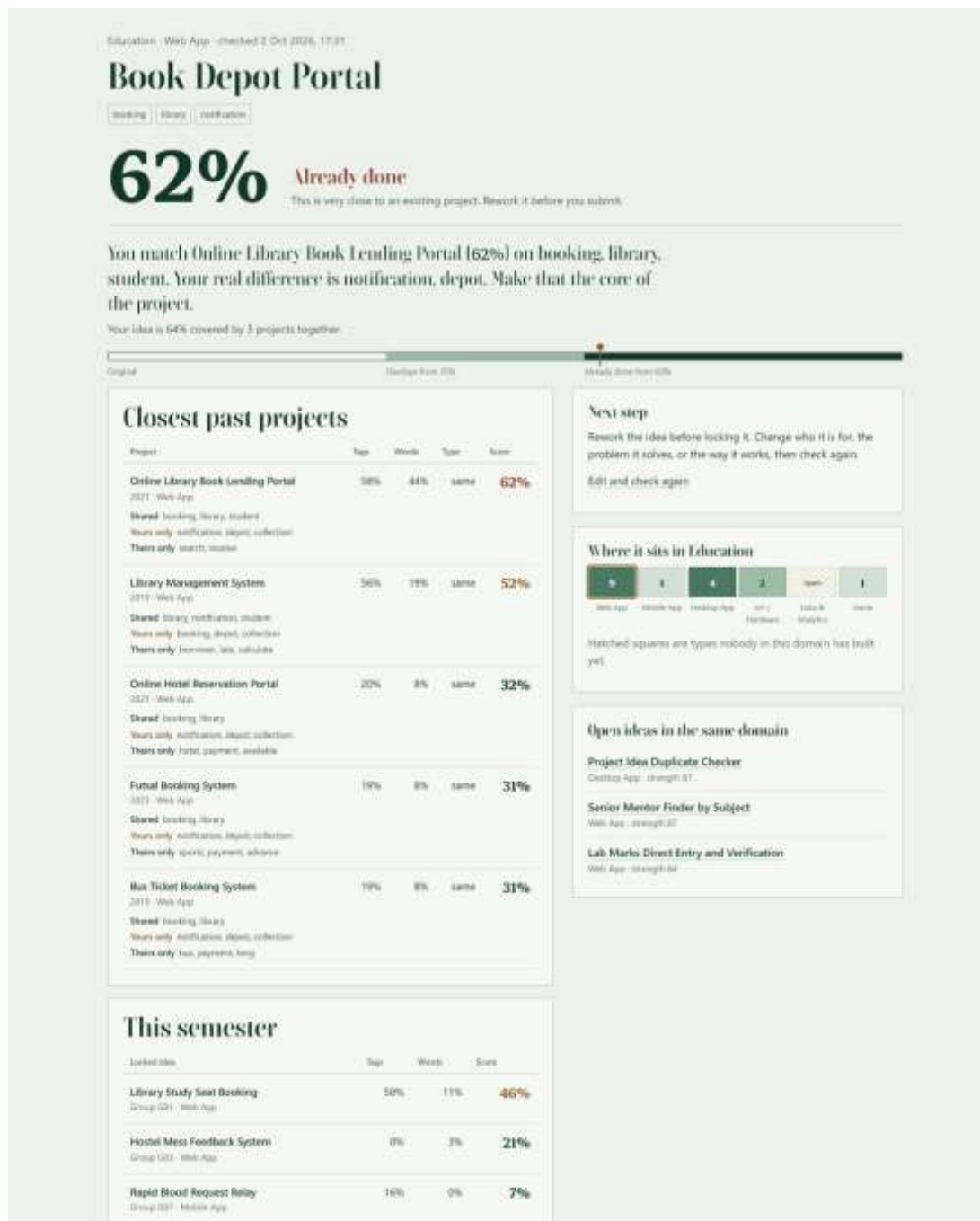


Figure 4.4: Screenshot - Renamed Library Idea Caught

Figure 4.5 shows the opposite case. A health-post medicine stock alert scores 30%, falls in the Looks original band, and the page offers to lock it for group G09. Its closest past project, the Student Attendance Management System, shares only a dashboard with it, and what only this idea has is medicine and inventory. The three closest projects together cover 17% of the idea.

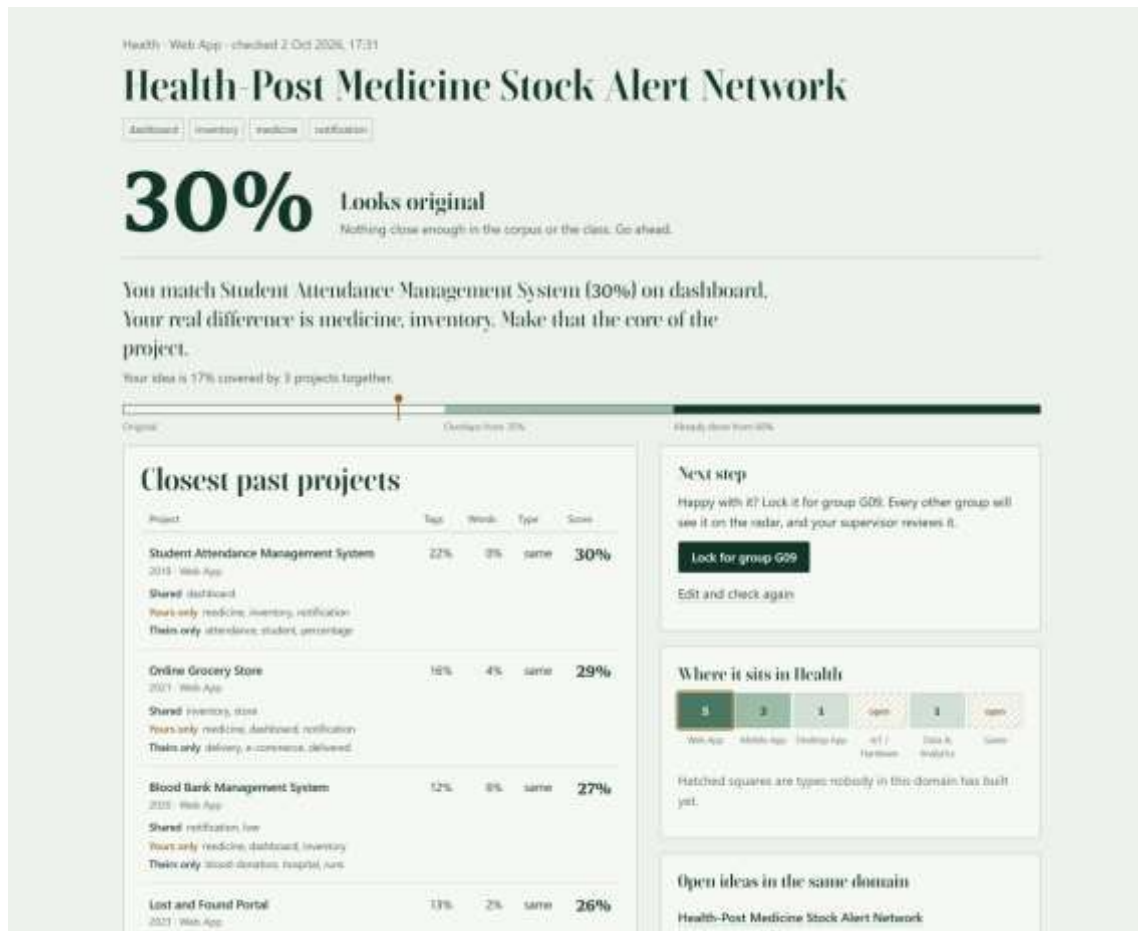


Figure 4.5: Screenshot - Original Idea Ready to Lock

4.2.4 Class Radar Module

RadarService locks an idea for the student's group. It refuses when the account has no group, when the verdict is Already done, or when the group has already locked an idea. The last rule is enforced twice: once in Java for a friendly message, and once by the unique key on the proposals table, which stops two teammates who click at the same moment. A lock always uses the final verdict. After the insert, the new idea is scored against every other locked idea, and every pair at 35% or above is written to radar_collisions in the same transaction. A trigger copies each clash into the audit trail. In the seed data, group G06's Verified Blood Request Line and group G07's Rapid Blood Request Relay clash at 41%, and both rows of Figure 4.6 show it.

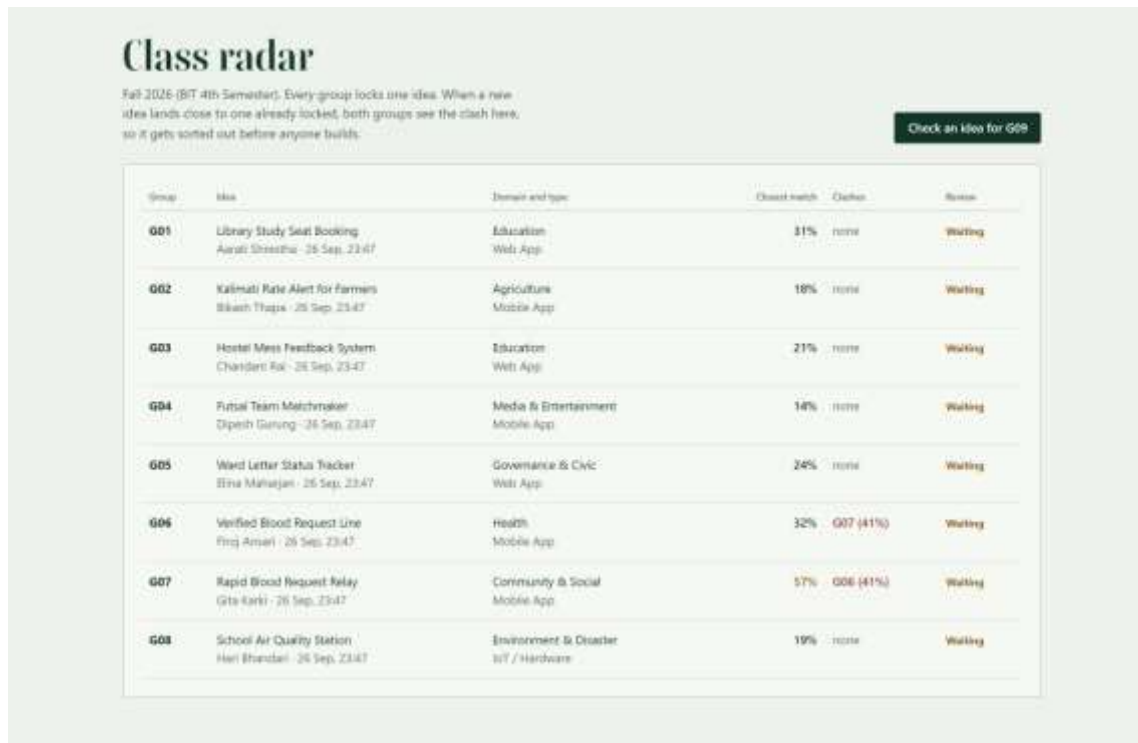


Figure 4.6: Screenshot - Class Radar with a Clash between G06 and G07

4.2.6 Syllabus Map and Idea Suggester Module

SyllabusService describes the syllabus as 41 topics (Java 10, Database 9, Operating Systems 9, Computer Organization 6, Statistics 7), each with a course, a unit and a few keywords. TopicTagger links a past project to every topic whose keywords appear in its title, description or tech stack. It reads two and three neighbouring words as one, so "round robin" matches the keyword round-robin, and it avoids everyday words such as "scan" or "view" that would mark ordinary projects. With the 71 demo projects, 16 of the 41 topics were never used. JDBC appears in 66 projects, Servlet and JSP in 40 and Swing in 15, while 8 of the 9 Operating Systems topics are open. The tech stacks of the demo projects were written for the demonstration.

The home page turns these counts into the Syllabus Map. Each plot is a topic, shaded by how many past projects used it and hatched when none did. As a visitor types, the page asks the /peek servlet about the text after a 300 millisecond pause, so a burst of typing makes one request. The servlet scores the text against the past projects through MatchIndex and returns the originality number, the closest project, the topics the text mentions and the topics of the close matches. Plots on worked ground turn red, topics the idea uses get an ochre ring, and the colour travels across the map as a wave. A library idea

reads Already done at 68% alike and turns five plots red (Figure 5.1). An idea built on the Banker's algorithm reads Looks original, and only the Deadlock plot is ringed (Figure 4.8).

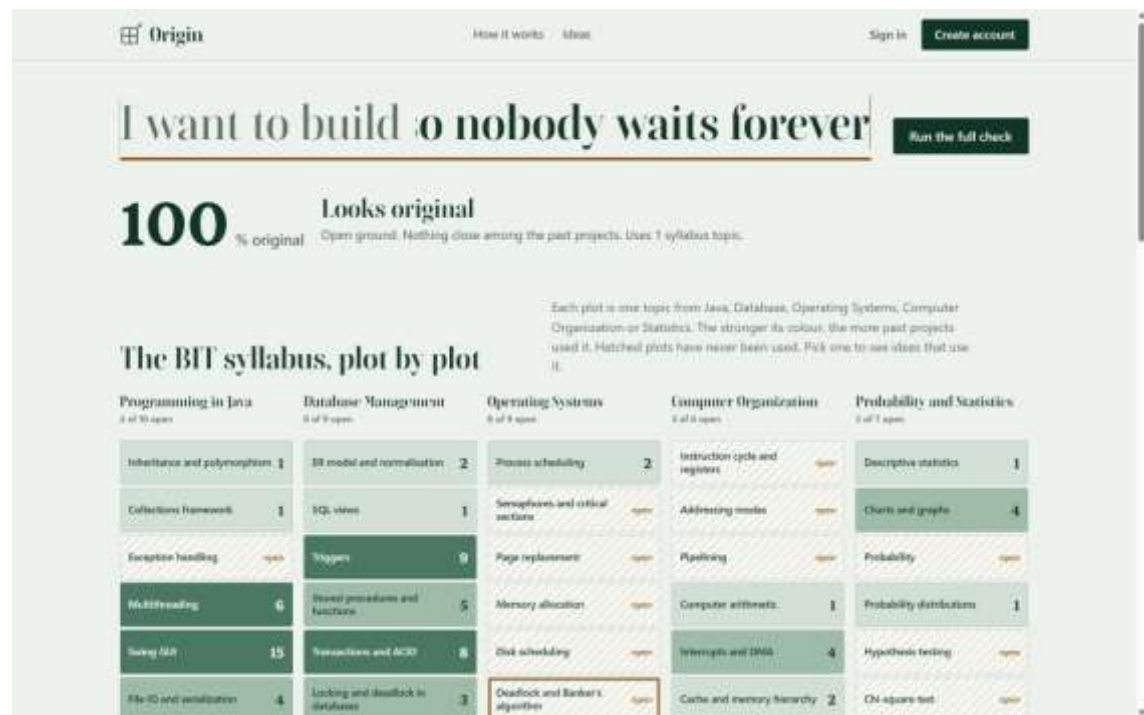


Figure 4.8: Screenshot - Syllabus Map on Open Ground

Choosing a plot filters the ideas below the map to those built on that topic (Figure 4.9), and Surprise me lights the topics of one strong idea. The page works without JavaScript: the map, the first ideas and the links are ordinary HTML.

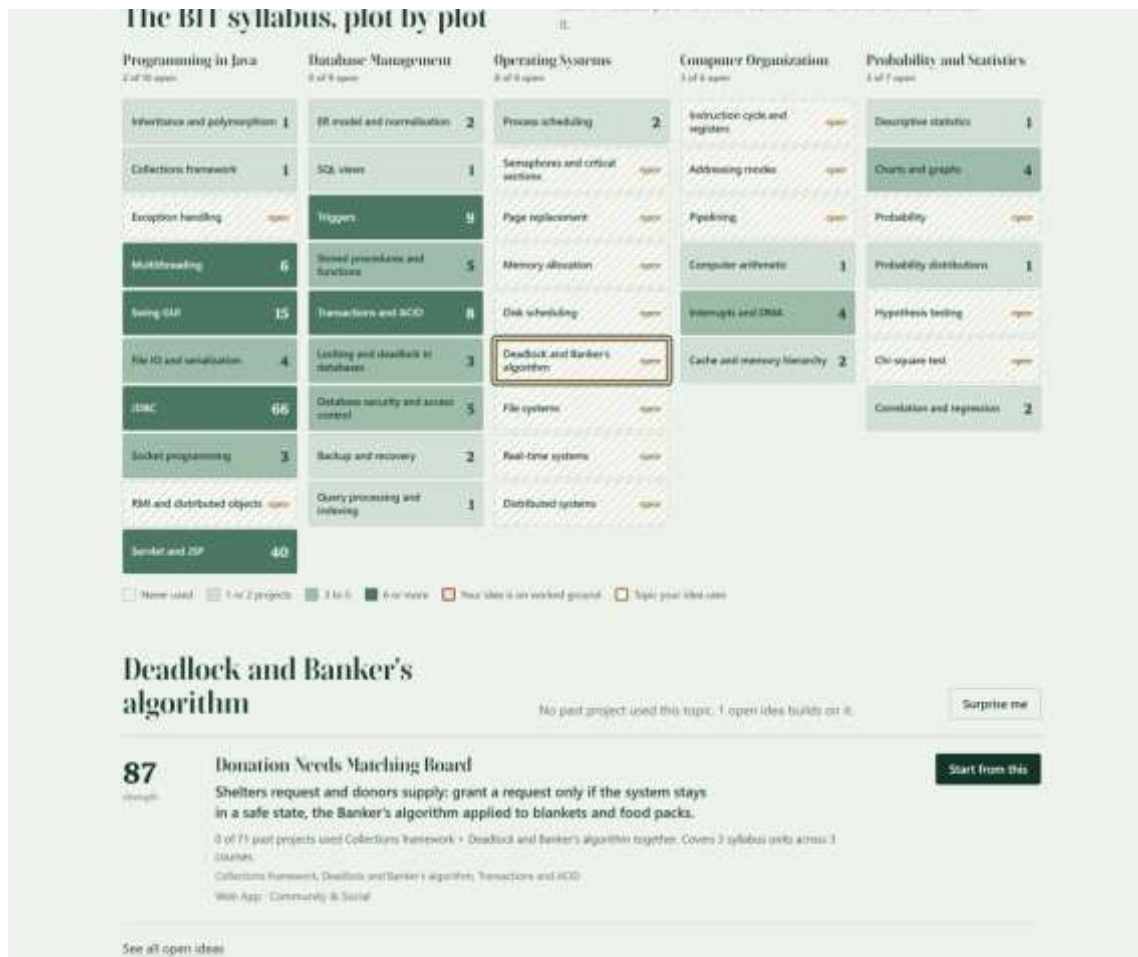


Figure 4.9: Screenshot - Plot Filter and the Idea Built on It

IdeaService pairs each of the 42 problems in the problem bank with its suitable project types, which gives 84 suggestions, and with two to four syllabus topics and a one-line twist written for it, such as using the Banker's algorithm to share blankets between shelters. A suggestion's strength is $0.40 \times \text{untouched} + 0.30 \times \text{coverage} + 0.20 \times \text{feasibility} + 0.10 \times \text{impact}$. Untouched is $1 / (1 + n)$, where n is how many past projects used the idea's busiest pair of topics together. Coverage is the number of syllabus units the idea uses out of four. Feasibility comes from the project type's effort and impact from the problem's source. An idea whose topics are all everyday plumbing (JDBC, Servlet and JSP, ER design) is flagged as plain CRUD and ranked after every other idea. An idea 60% or more like a past project is retired, and one that a locked idea already matches at 45% or more is marked as taken by that group. Every suggestion shows its proof, for example "0 of 71 past projects used Descriptive statistics + Multithreading together. Covers 4 syllabus units across 3 courses." Of the 84 suggestions, 77 are open and 7 are taken. Figure 4.10 shows the Ideas page.

Ideas for the next project

42 real problems from Nepal, each built from syllabus topics. Ranked by strength: how untouched its topic mix is, how much of the syllabus it uses, how buildable in one semester and how useful. Plain CRUD ideas come last. Another 8 problems reported by students this semester join the list when the semester closes.

Domain: All domains
Type: All types
Syllabus topic: Any topic
Difficulty: Any
☐ Show hidden ideas
Filter

94 strength

Allowance Payment SMS Notifier

Sending 10,000 SMS in one batch needs a worker pool: chart throughput as the thread count grows and show where more threads stop helping.

Social security allowance beneficiaries are not informed when payments are released.

8 of 71 past projects used Descriptive statistics + Multithreading together. Covers 4 syllabus units across 3 courses.

Descriptive statistics, Multithreading, Charts and graphs, Smartphones and critical sections

Desktop App - Governance & Civic - for elderly and simple-worried beneficiaries

Closest existing work: Eyes Membership Management, 27% alike, 1st part Desktop App in this domain

Start from this

How: 70
Buildable: 85
Useful: 70
Easy

94 strength

Soil Test Result Advisor

Compare a field's nitrogen reading with the district mean using a one-sample t-test, and advise fertilizer only when the difference is significant.

Farmers cannot tell which fertilizer their soil needs and overuse urea.

8 of 71 past projects used Descriptive statistics + Probability distributions together. Covers 4 syllabus units across 2 courses.

Descriptive statistics, Probability distributions, Hypothesis testing, SQL views

Web App - Agriculture - for Paddy and maize farmers

Closest existing work: Eritre Sugar Online Marketplace, 22% alike, 1 part Web App in this domain

Start from this

How: 70
Buildable: 85
Useful: 70
Easy

94 strength

Local Folk Music Archive

Store the audio files in a directory scheme you design like a small file system, with an index for search and an LRU cache for the most played songs.

Local folk musicians have no place to publish and be discovered by listeners.

8 of 71 past projects used Query processing and indexing + File IO and serialization together. Covers 4 syllabus units across 3 courses.

Query processing and indexing, File IO and serialization, Page replacement, File systems

Web App - Media & Entertainment - for Folk and local musicians

Closest existing work: Music Streaming App, 30% alike, 3 part Web App in this domain

Start from this

How: 70
Buildable: 85
Useful: 70
Easy

Figure 4.10: Screenshot - Ranked Project Ideas with Proof

4.2.7 DB Folder Import Module

A watcher thread looks in the DB folder every five seconds. A file is read once its size and time have stayed the same for one scan, so a file that is still being copied is never half-read, and a changed file is read again. TableReader picks a reader by the file extension: DelimitedReader for CSV and TXT, which finds the separator in the header line and handles quoted cells, and XlsxReader and DocxReader for the zip-based formats. The XML parsers have DTDs and external entities switched off, so a hostile file cannot read other files from the server. HeaderMap finds columns by header name, ignoring case and spacing, so "Project Name", "project_name" and "Title" are the same column.

IngestService adds all the new rows of a file in one transaction: either every row goes in or none does. Rows already in the corpus (same title and year) are skipped, and a row without a title or a valid year is skipped with its line number. A missing domain or type is worked out from the text by Classifier, and missing tags from the vocabulary. Each file version gets one line in ingest_log, which the supervisor page shows. A file the system cannot read, such as a PDF, is listed as unsupported with the advice to convert it. In a test

with a real Excel file and a CSV, both were read 8.0 seconds after being copied: the workbook added 3 projects, and the CSV added 1, skipped 1 duplicate and reported a row with no year by its line number.

4.2.8 Supervisor Desk and Self-Updating Database

The supervisor desk (Figure 4.11) lists the locked ideas waiting for review with their clashes, the DB folder with the log of the files it has read, the share of past projects per domain from the v_domain_saturation view, the tags students check most, and the audit trail. On any result page, a supervisor can mark a match as the same project or a different one. The correction is stored against the idea's fingerprint, a SHA-256 hash of its sorted tags and keywords, so the same idea checked again later receives the corrected verdict without asking anyone.

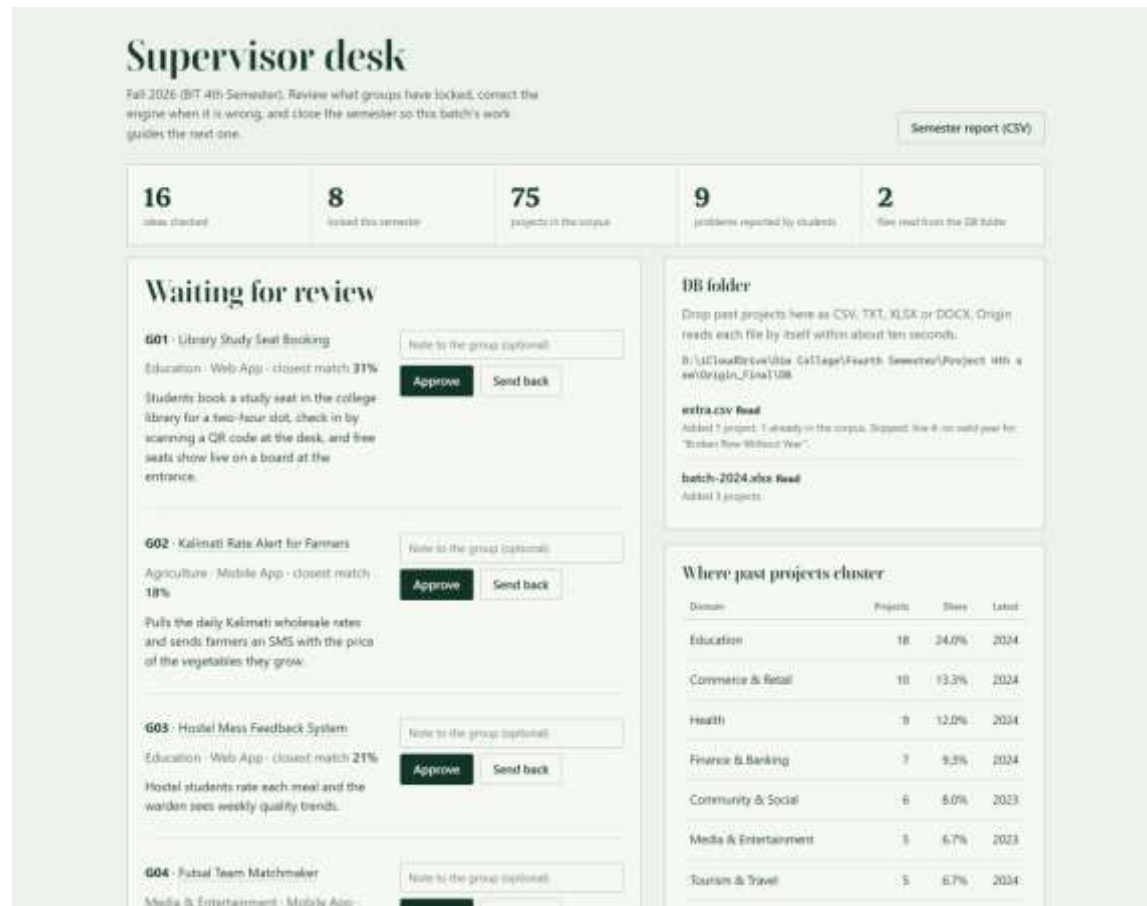


Figure 4.11: Screenshot - Supervisor Desk

The review comments before the proposal defense asked for more active use of the database. Table 4.3 lists the database objects that now do work on their own.

Table 4.3: Database Objects that Keep Origin Current

Object	Type	What it does
trg_cpt_count, trg_ict_count	Triggers	Keep per-tag counts for the corpus and for checks; copy a student problem's tags into the problem bank
trg_check_problem	Trigger	Copies a student's problem statement into the problem bank
trg_proposal_locked, trg_proposal_reviewed, trg_collision_audit	Triggers	Write every lock, review and clash to the audit log
v_domain_saturation, v_class_radar, v_check_history	Views	Supply statistics, radar and history pages
fn_band	Stored function	Turns a score into a band with the same thresholds as the Java code
sp_archive_semester	Stored procedure	In one transaction: copies approved ideas and tags into the corpus, closes the semester, opens the next
sp_semester_report	Stored procedure	Returns one row per group for the supervisor's CSV report
ingest_log	Table	Remembers which version of which DB-folder file has been read, so an unchanged file is never read twice
syllabus_topics, corpus_project_topics, problem_topics	Tables	Hold the 41 topics and link past projects and problems to them; the map and the proof lines are counted from these

4.3 Testing

4.3.1 Testing Strategy

We tested at three levels. Unit tests with JUnit 5 cover the scorers, the term weights, the overlap, coverage and story, the inverted index, the three file readers, the topic tagger, the suggester's scoring, the vocabulary, the claim detector, the password hasher, the CSV writer and the SQL script splitter. Integration tests run the real services against separate MySQL databases named `origin_test` and `origin_ingest_test`, which are dropped and rebuilt on every run, so they exercise the triggers, views, constraints, stored procedures and the folder watcher with real files. System tests drove the running application in Chrome from a Node script: typing into the home page, clicking, submitting checks, dropping real files

into the DB folder and recording the console. HTTP checks with curl covered the security rules.

The accuracy of the formula was tested against human judgement. Before running it, the team labelled 25 pairs of ideas as Looks original, Overlaps or Already done: the 15 used for the first version and 10 that stress small ideas, renames and name-only collisions. The first version of the new formula agreed on 19 of the 25 pairs while the old formula agreed on 22, so it was not accepted. A sweep over the share given to the second overlap measure showed that giving it half of the overlap cost three pairs, so the share was cut to 20% and the original weights were kept. The accepted formula agrees on 22 of 25, the same as the old one, and on 13 of the original 15. The test fails if the new formula agrees on fewer pairs than the old one, or on fewer than 12 of the original 15.

4.3.2 Test Cases

The following test cases (Table 4.4) document the testing carried out on the system. Each test case verifies a specific functionality against expected behavior.

Table 4.4: Test Cases

TC ID	Test Description	Input Data	Expected Output	Actual Output	Status	Type
TC-01	Valid user login	sandesh@origin.edu, correct password	Redirect to Check an idea	HTTP 302 to /app/check	Pass	System
TC-02	Invalid login credentials	Correct email, wrong password	Error message, no session	"That email and password do not match an account."	Pass	System
TC-03	Repeated failed logins	Six wrong passwords in a row	Sixth attempt refused	HTTP 429 on the sixth attempt	Pass	Security
TC-04	Registration with invalid data	Name "A", bad email, short password, group "7"	Error on each field	4 field errors returned	Pass	Unit
TC-05	Registration with duplicate email	sandesh@origin.edu	Duplicate email error shown	"An account with this email already exists."	Pass	Unit
TC-06	Empty or short fields in the check form	Title "Hi", 10-character description, one tag	Errors on title, description and tags	3 field errors, nothing saved	Pass	Integration
TC-07	Check - renamed library idea	"Book Depot Portal" with books, lending, reservation	Already done; closest match is a library project	DUPLICATE (62%); top match Online Library Book Lending Portal	Pass	Integration

TC ID	Test Description	Input Data	Expected Output	Actual Output	Status	Type
TC-08	Lock a duplicate idea	The check from TC-07	Lock refused	"This idea is marked as already done..."	Pass	Integration
TC-09	Check and lock an original idea	Livestock Vaccination Reminder	Looks original; proposal created	ORIGINAL; proposal saved for G09	Pass	Integration
TC-10	Second lock by the same group	Any check by a G09 member	Lock refused	"Group G09 has already locked an idea this semester."	Pass	Integration
TC-11	Buzzword padding	"AI Powered Hostel Prediction System"	Claims flagged with reasons	Claim flag set, 2 reasons shown	Pass	Integration
TC-12	Background refinement (optional engine, simulated)	Simulated engine answering 0.95	Status DONE; final = 0.6 rule + 0.4 x 0.95	Status DONE; final score matched to 0.001	Pass	Integration
TC-13	Supervisor correction remembered	Mark a match "Different", check the same idea again	Correction applied without asking again	Meaning score 0.0 on the second check	Pass	Integration
TC-14	Triggers keep counters and audit	Seed data for 8 groups	Tag counts equal link rows; 8 LOCK rows	Counts equal; 8 LOCK and 1 COLLISION rows	Pass	Integration
TC-15	Close the semester	Approve G02, call sp_archive_semester	1 project moved; radar empty; new semester active	1 moved; 0 proposals; 1 active semester	Pass	Integration
TC-16	Student opens the supervisor desk	Student session, GET /admin	Access refused	HTTP 403 "Supervisors only"	Pass	Security
TC-17	Form posted without CSRF token	POST /login without _csrf	Request refused	HTTP 403 "Form expired"	Pass	Security
TC-18	Another student's result	GET /app/result?id=1 as another student	Not shown	HTTP 404	Pass	Security
TC-19	Export check history	GET /app/history.csv	CSV file download	text/csv attachment with a header row	Pass	System
TC-20	Agreement with human labels	25 labelled idea pairs	New formula agrees at least as often as the old one	22 of 25 (old formula: 22 of 25)	Pass	Unit
TC-21	Lock before the background check finishes (optional engine, simulated)	Pending check; engine says same project (0.99)	Comparison runs first, then the lock is refused	Status DONE; "This idea is marked as already done..."	Pass	Integration

TC ID	Test Description	Input Data	Expected Output	Actual Output	Status	Type
TC-22	Rapid typing on the home page	A 60-character idea typed key by key	One request to /peek	1 request	Pass	System
TC-23	Map reacts to a typed idea	A library idea with fines and reminders, then a Banker's algorithm idea	Worked ground for the first, open ground for the second	68% alike, Already done, 5 plots red; then 100% original, 1 plot ringed	Pass	System
TC-24	Real Excel file dropped in the DB folder	batch-2024.xlsx written by Excel, 3 projects	Projects appear within about 10 seconds	3 projects added, visible after 8.0 seconds	Pass	System
TC-25	CSV with a duplicate and a bad row	3 rows: new, existing title and year, no year	1 added, 2 skipped with reasons	1 added; 1 already in the corpus; line 4 has no valid year	Pass	System
TC-26	File still being copied	A file that grows between scans	Not read until its size stops changing	Read on the third scan	Pass	Integration
TC-27	Hostile workbook	XLSX whose XML declares an external entity	Entity not expanded	No secret text in the result	Pass	Unit
TC-28	Proof matches a hand count	All 84 suggestions	Pair count equals a SQL count of projects using both topics	84 of 84 equal	Pass	Integration
TC-29	Plain CRUD ranked last	Suggestions whose topics are only JDBC, Servlet or ER design	Never listed above a real idea	No real idea listed after a CRUD one	Pass	Integration
TC-30	Evaluation screen	Submit a check	Five stages with real counts, then the result	Stages run in about 2.4 s; result opens about 2.9 s after the screen appears; reduced motion goes straight to the result	Pass	System
TC-31	Live preview throttle	40 calls from one address in a few seconds	Calls beyond 30 refused	30 answered, 10 refused with HTTP 429	Pass	Security
TC-32	Phone width	Home, Ideas, History and Radar at 390 px	No sideways page scroll	None	Pass	System

Note: All test cases were run on the final build. No test case failed in the final run; the failure found during development is described in Section 4.3.3.

4.3.3 Test Results Summary

- Total automated tests executed: 84 (59 unit, 25 integration), plus scripted Chrome runs that typed, clicked, dropped files and captured 20 pages on desktop and phone, with no console error in any run.
- Tests passed: 84 of 84, and every HTTP check returned the expected status.
- Failed during development: 3. First, the rename test scored 0.5 instead of at least 0.66 for tags, because plural tags such as "books" were not folded to their synonym; the fix folds a plural tag only when its singular is a known tag or synonym. Second, the new matching formula agreed with 19 of 25 labelled pairs against 22 for the old one; reducing the second overlap measure to 20% fixed it. Third, the first Syllabus Map had 37 of 41 plots open, because the demo abstracts say what a project does and not how it was built; storing the tech stack with each project gave 16 open plots.
- Tools used: JUnit 5, Maven Surefire, separate MySQL test databases, curl and a Chrome DevTools script in Node.

CHAPTER 5: FINAL OUTCOME

5.1 System Output and Analysis

The final system runs on Apache Tomcat with MySQL and serves twelve pages to two roles. The landing page opens on one prompt, "I want to build", over the Syllabus Map (Figure 5.1). Typing a library idea turns the readout to Already done and five plots red; typing an idea built on the Banker's algorithm leaves the map cool and reads Looks original. With the demo data, 16 of the 41 topics have never been used by a past project.

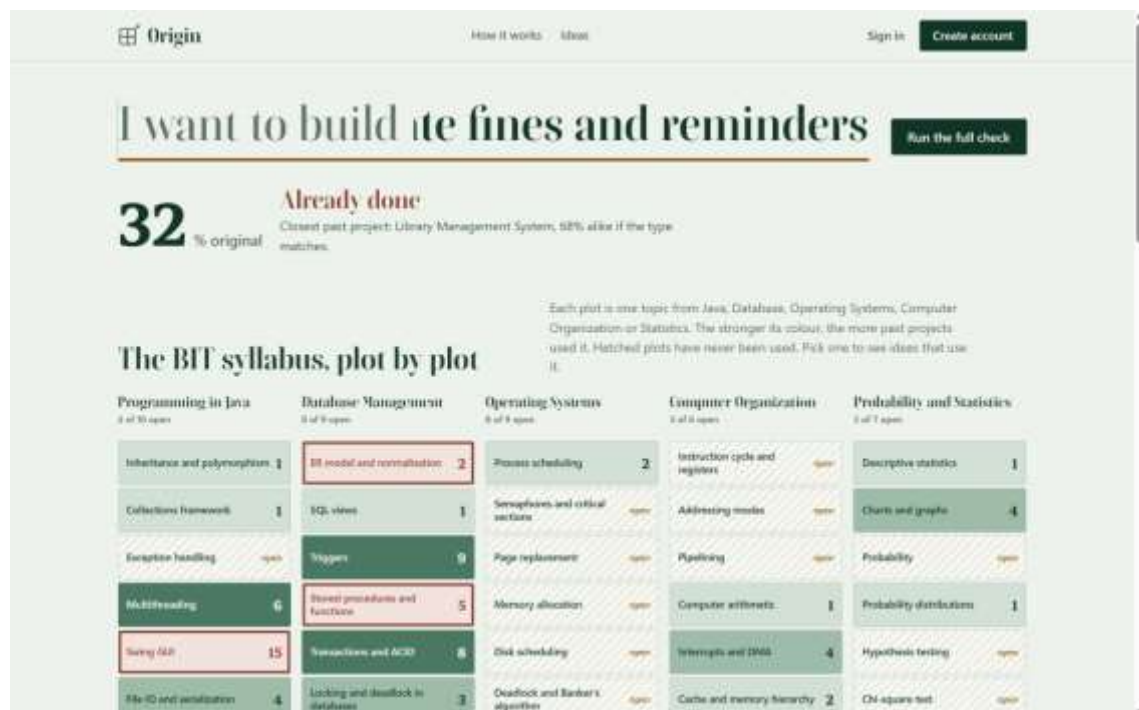


Figure 5.1: Screenshot - Landing Page with the Syllabus Map

Further down, the landing page runs the rename test through the live engine every time it loads (Figure 5.2). "Book Depot Portal" is matched to the Online Library Book Lending Portal with 77% of tags and 61% of words in common and the same type, for 76% overall. The number is computed, not typed, so it stays true if the corpus changes.

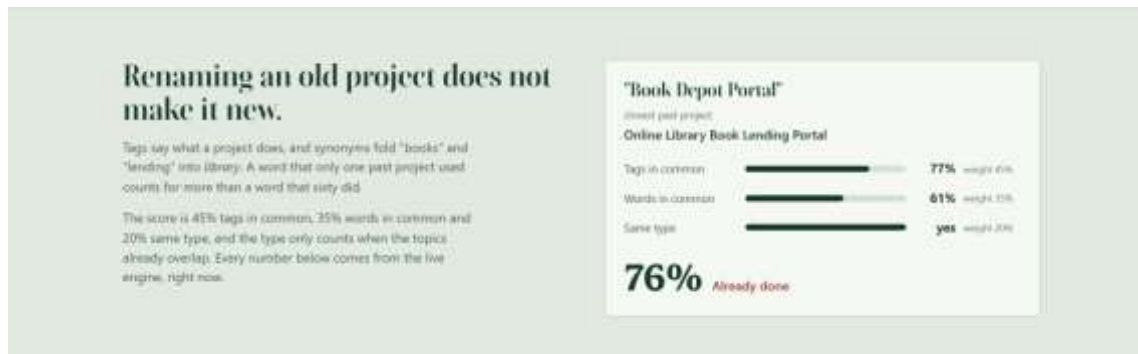


Figure 5.2: Screenshot - Rename Example Computed by the Live Engine

5.2 Analysis of Output Against Objectives

Objective 1: To analyse how BIT project ideas repeat and to study existing originality and plagiarism tools.

Outcome: Seven related systems were studied (Table 2.1); all of them check finished work. The sample corpus of 71 typical projects shows the pattern: 40 are web applications and education alone holds 15, while 40 of the 72 domain and type combinations have no project at all.

Objective 2: To design a layered architecture and a normalised schema that keeps itself current.

Outcome: The system uses three layers (Figure 3.2) and 25 tables in third normal form (Figure 3.5). Six triggers, four views, one stored function and two stored procedures do work on their own (Table 4.3). TC-14 and TC-15 prove the triggers and the archive procedure against a real database.

Objective 3: To develop an explainable similarity engine that resists renaming and buzzword padding.

Outcome: Every score on the result page is split into tags, words and type and told as a story (Figure 4.4). The renamed library idea is caught at 62% (TC-07) and padding is flagged (TC-11). The new formula agrees with the team's human labels on 22 of 25 pairs, as often as the old one (TC-20).

Objective 4: To develop a Syllabus Map and an idea suggester.

Outcome: The map shows all 41 syllabus topics with their counts and filters the ideas (Figures 4.8 and 4.9). The suggester turns 42 problems into 84 scored suggestions with proof lines: 77 are open, 7 are marked as taken by locked groups, and none is plain CRUD (Figure 4.10).

Objective 5: To implement Class Radar, supervisor review, semester archiving and the DB folder.

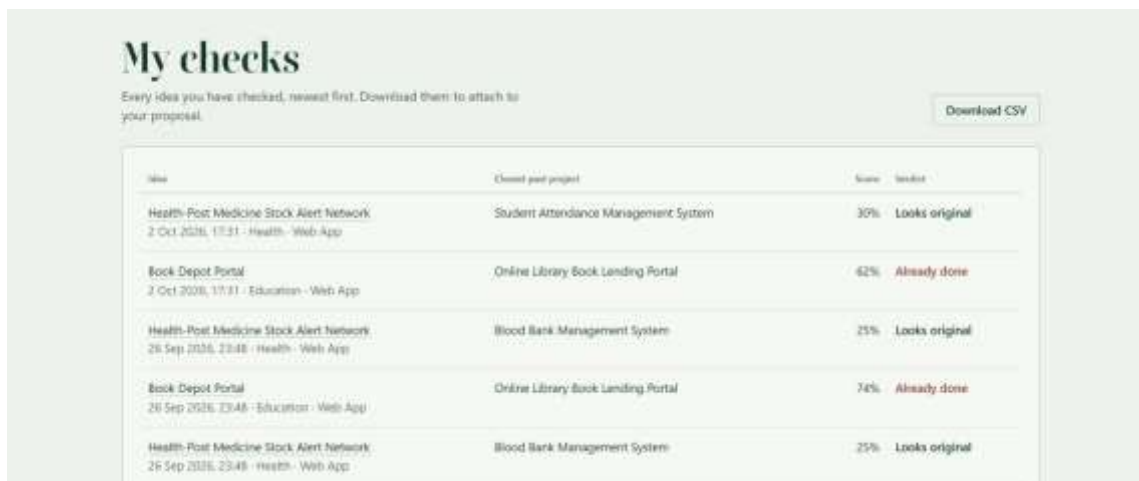
Outcome: Locking, clash detection and review work end to end (Figures 4.6 and 4.11; TC-09, TC-10). The G06 and G07 clash is detected at 41%. A real Excel file and a CSV dropped into the DB folder were read within 8.0 seconds, with the duplicate skipped and the bad row reported (TC-24, TC-25).

Objective 6: To test the system for correctness, security and performance.

Outcome: All 84 automated tests pass, the security rules hold over real HTTP (TC-16 to TC-18), and the timings in Section 5.4 are well inside the two-second requirement.

5.3 Final System Screenshots

Figure 5.3 shows a student's check history, which can be downloaded as CSV for a proposal, and Figure 5.4 shows the same pages on a phone screen. No page scrolls sideways as a whole at a 390-pixel width.



My checks
Every idea you have checked, newest first. Download them to attach to your proposal.

[Download CSV](#)

Name	Checked post project	Score	Status
Health-Post Medicine Stock Alert Network 2 Oct 2026, 11:21 - Health - Web App	Student Attendance Management System	30%	Looks original
Book Depot Portal 2 Oct 2026, 11:21 - Education - Web App	Online Library Book Lending Portal	62%	Already done
Health-Post Medicine Stock Alert Network 26 Sep 2026, 23:48 - Health - Web App	Blood Bank Management System	25%	Looks original
Book Depot Portal 26 Sep 2026, 23:48 - Education - Web App	Online Library Book Lending Portal	76%	Already done
Health-Post Medicine Stock Alert Network 26 Sep 2026, 23:48 - Health - Web App	Blood Bank Management System	25%	Looks original

Figure 5.3: Screenshot - Check History with CSV Export

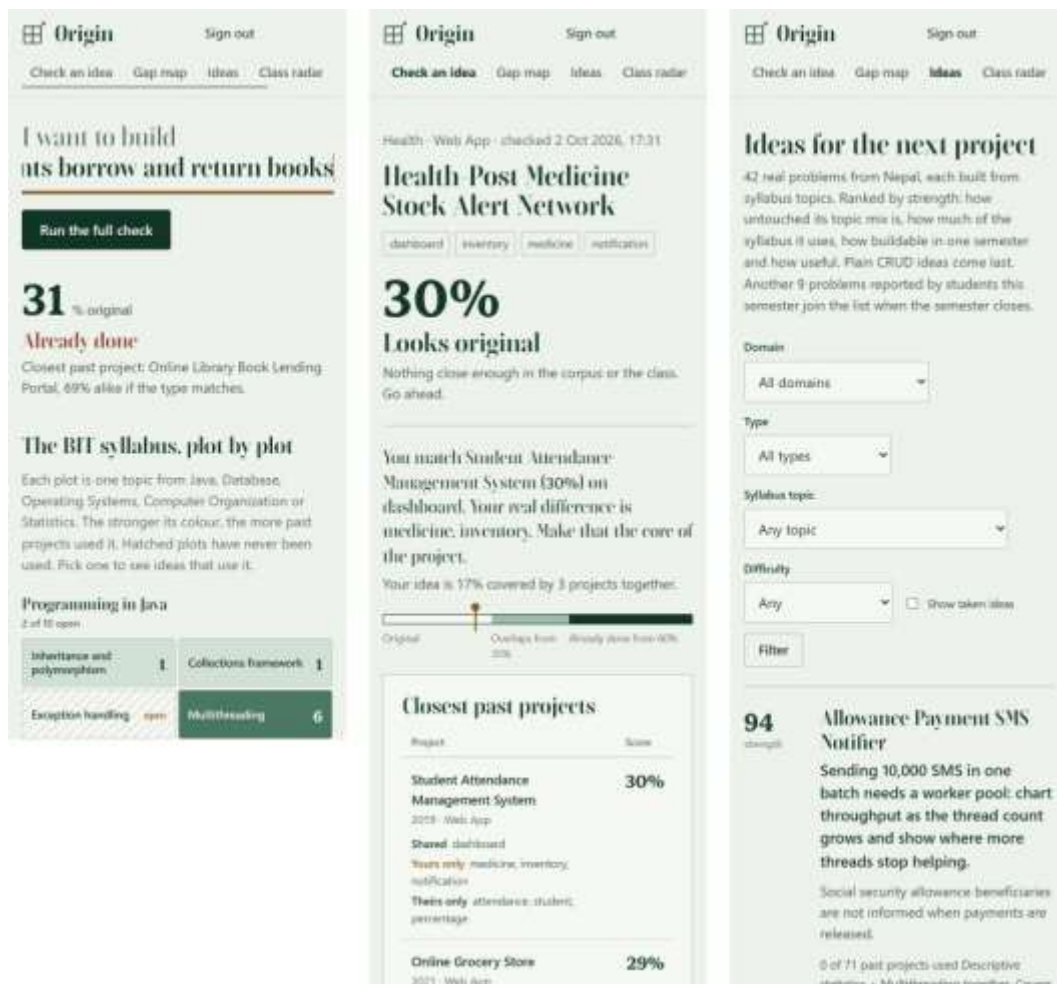


Figure 5.4: Screenshot - Landing, Result and Ideas Pages on a Phone

5.4 Performance and Usability Observations

We timed the running system on a development laptop with MySQL on the same machine, averaging seven requests per page after one warm-up request, while other programs were using the laptop. Table 5.1 shows the results.

Table 5.1: Measured Response Times

Request	Average time
Landing page with the Syllabus Map	376 ms
Live preview while typing (/peek)	66 ms
Ideas page (first 15 of 42)	314 ms
Class Radar	149 ms
Check form	273 ms
Submit a check (validate, score, save in one transaction)	328 ms
Evaluation screen	148 ms
Result page with matches told as stories	272 ms

Every request finished in under 0.4 seconds, far inside the two-second requirement. Each request opens its own database connection, which accounts for most of the time. The live preview that runs while a visitor types takes about 65 milliseconds. A check scores only the projects that share a term with the idea, because MatchIndex looks them up instead of scanning the corpus.

For usability, we checked the pages at desktop and phone widths and in a dark colour scheme with scripted Chrome runs, and fixed every problem we found before release. No page scrolls sideways at 390 pixels. The map plots are real buttons, so the keyboard works, and the evaluation screen has a plain link to skip ahead. Every page works without JavaScript: the scripts add the live map reaction and the evaluation stages, but no content depends on them, and a visitor who prefers reduced motion gets no animation.

CHAPTER 6: CONCLUSION, LIMITATIONS & FUTURE RECOMMENDATIONS

6.1 Conclusion

This project set out to give BIT students a way to check a project idea before committing a semester to it. Origin does that with Java Servlets, JSP and JDBC on Apache Tomcat and a MySQL database. It compares an idea with the college's past projects and with the ideas other groups have locked, explains every score, catches renamed ideas and unsupported buzzwords, and shows each group's clashes on the Class Radar.

It also answers the question that follows a failed check. The Syllabus Map shows which topics of the BIT syllabus no past project has used, and the Ideas page ranks real problems from Nepal that are built from those topics, with a proof line for each. Past projects load by themselves from files dropped into a folder.

The database does part of the work. Triggers maintain counters and the audit trail, a stored procedure moves each semester's approved ideas into the corpus in one transaction, and a background thread keeps the suggestions current. All six specific objectives were met, as Section 5.2 shows, with all 84 automated tests passing. The new matching formula agrees with human labels as often as the old one, so its gain is in how clearly it explains a match and not in accuracy. The result is a tool that helps students and supervisors have a better conversation about ideas, earlier in the semester.

6.2 Problems Faced and Limitations

Challenges Encountered:

- Seed data that the engine rejected. Two of our own seeded group ideas were close copies of corpus projects, and the engine refused to lock them on the first run. We rewrote those seed ideas; the engine was right.
- Embedded Tomcat under Maven. Tomcat could not load its own classes through Maven's exec plugin. We launch it as a plain Java process with the full classpath instead, which is also simpler for the demonstration.
- A display font with old-style figures. The chosen display face drew the digit 1 like the letter I, which confused the map. We combined two font files so that letters and digits come from different faces.

- Plural tags. "books" was not folded to library at first (Section 4.3.3). Folding plurals only for known words fixed it without damaging tags such as "analytics".
- A formula that did worse than the old one. The first version of the new matching formula agreed with 19 of 25 labelled pairs against 22 for the old one, because counting how much of a small idea an old project already has pushed overlapping ideas into the Already done band. A sweep showed the cause and the share was cut to 20%. Accuracy is now equal, not better.
- A map that showed nothing. The first Syllabus Map had 37 of 41 plots hatched, because the demo abstracts say what a project does and not how it was built. We stored the tech stack with each project and read topics from it, which gave 16 open plots and a spread that matches what students actually build.

Current Limitations:

- The corpus and problem bank are seed data prepared by the team, and the tech stacks of the 71 demo projects were written for the demonstration. The college's real archive has to be dropped into the DB folder before Origin can judge real proposals.
- Scores depend on sensible tags. A student who tags carelessly weakens the comparison.
- Topic detection is keyword matching, so it can miss a topic that a project used without naming it and can count one that it only mentions. The type counts 20% of a score, so a project of the same type that shares one tag can outrank a closer project of another type: the health-post idea in Figure 4.5 does not list the pharmacy desktop application among its five closest projects.
- Origin is designed for one college and one programme at a time.
- It compares ideas, not writing or code, and does not replace tools such as Turnitin for finished reports.
- PDF files cannot be read. A real Excel file, a CSV and a Word table were each dropped into the running app and imported correctly.
- Every request opens its own database connection, which is simple but not the fastest design.
- An optional second opinion from an external decision engine exists in the code and is switched off, so the system runs on rules alone.

6.3 Future Recommendations

- Add an upload button to the supervisor desk and read PDF files, so the college's old reports can be loaded without conversion.
- Let supervisors add and edit problems in the problem bank from the supervisor desk, including problems sent by local organisations.
- Send an email to both groups when a clash is detected on the Class Radar.
- Measure agreement with human labels on a larger set of pairs each semester and tune the weights from the results.
- Support several programmes and colleges, each with its own corpus and radar.
- Add a view of how each domain's share of projects changes over the years.
- Switch the optional decision-engine second opinion back on and test it with a live key [11].
- Use a connection pool, which is outside the current syllabus, and measure how much time it saves.

REFERENCES

- [1] Turnitin, LLC, "Turnitin Similarity," Turnitin. [Online]. Available: <https://www.turnitin.com/products/similarity>. [Accessed: 26 Sep. 2026].
- [2] S. Schleimer, D. S. Wilkerson, and A. Aiken, "Winnowing: Local algorithms for document fingerprinting," in Proc. ACM SIGMOD Int. Conf. Management of Data, San Diego, CA, USA, 2003, pp. 76-85.
- [3] P. Jaccard, "The distribution of the flora in the alpine zone," *New Phytologist*, vol. 11, no. 2, pp. 37-50, Feb. 1912.
- [4] C. D. Manning, P. Raghavan, and H. Schütze, *Introduction to Information Retrieval*. Cambridge, U.K.: Cambridge University Press, 2008.
- [5] A. Silberschatz, H. F. Korth, and S. Sudarshan, *Database System Concepts*, 7th ed. New York, NY, USA: McGraw-Hill, 2019.
- [6] Oracle Corporation, "MySQL 8.4 Reference Manual: Stored Objects," MySQL. [Online]. Available: <https://dev.mysql.com/doc/refman/8.4/en/stored-objects.html>. [Accessed: 26 Sep. 2026].
- [7] Eclipse Foundation, "Jakarta Servlet Specification, Version 6.0," Jakarta EE. [Online]. Available: <https://jakarta.ee/specifications/servlet/6.0/>. [Accessed: 26 Sep. 2026].
- [8] The Apache Software Foundation, "Apache Tomcat 10.1 Documentation," Version 10.1. [Online]. Available: <https://tomcat.apache.org/tomcat-10.1-doc/>. [Accessed: 26 Sep. 2026].
- [9] K. Moriarty, B. Kaliski, and A. Rusch, "PKCS #5: Password-based cryptography specification version 2.1," Internet Engineering Task Force, RFC 8018, Jan. 2017.
- [10] OWASP Foundation, "Password Storage Cheat Sheet," OWASP Cheat Sheet Series. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html. [Accessed: 26 Sep. 2026].
- [11] D. Almeida, "Introducing System One Models and Jev," TypeSafe AI, Sep. 14, 2026. [Online]. Available: <https://typesafe.ai/blog/introducing-system-one-models-and-jev>. [Accessed: 26 Sep. 2026].
- [12] Turnitin, LLC, "iThenticate," iThenticate. [Online]. Available: <https://www.ithenticate.com>. [Accessed: 26 Sep. 2026].
- [13] L. Prechelt, G. Malpohl, and M. Philippsen, "Finding plagiarisms among a set of programs with JPlag," *Journal of Universal Computer Science*, vol. 8, no. 11, pp. 1016-1038, Nov. 2002.
- [14] LYRISIS, "DSpace," DSpace. [Online]. Available: <https://dspace.lyrasis.org>. [Accessed: 26 Sep. 2026].
- [15] Google, "Google Scholar," Google. [Online]. Available: <https://scholar.google.com>. [Accessed: 26 Sep. 2026].
- [16] I. Sommerville, *Software Engineering*, 10th ed. Harlow, U.K.: Pearson, 2016.
- [17] H. Schildt, *Java: The Complete Reference*, 12th ed. New York, NY, USA: McGraw-Hill, 2021.
- [18] R. Elmasri and S. B. Navathe, *Fundamentals of Database Systems*, 7th ed. Hoboken, NJ, USA: Pearson, 2016.

ANNEX

Annex A: Source Code

The complete source code, the SQL schema, the seed data and the test suite are in the project repository.

GitHub Repository: [GitHub repository link]

The listings below are the parts most often asked about: the engine loop, the weighted overlap, the folder watcher, the trigger that grows the problem bank, and the archive procedure.

A.1 SimilarityEngine.compare (Java)

```
public Breakdown compare(Candidate idea, Candidate other) {
    double t = tags.score(idea, other);
    double k = keywords.score(idea, other);
    double y = type.score(idea, other);
    double total = tags.weight() * t + keywords.weight() * k;
    // the type counts only when tags or words already overlap
    if (t > 0 || k > 0) total += type.weight() * y;
    return new Breakdown(other, t, k, y, total);
}
```

A.2 Trigger: a student's problem joins the problem bank (MySQL)

```
CREATE TRIGGER trg_check_problem
AFTER INSERT ON idea_checks FOR EACH ROW
BEGIN
    IF NEW.problem IS NOT NULL
        AND CHAR_LENGTH(TRIM(NEW.problem)) >= 25 THEN
        INSERT IGNORE INTO problem_bank (domain_id, statement,
            affected, solution_phrase, primary_type_id, source)
        VALUES (NEW.domain_id, TRIM(NEW.problem),
            'Reported by a student', NEW.title, NEW.type_id,
            'STUDENT');
    END IF;
END
```

A.3 Stored procedure: close the semester (MySQL, abridged)

```
CREATE PROCEDURE sp_archive_semester(
    IN p_new_code VARCHAR(20), IN p_new_label VARCHAR(60),
    IN p_actor INT, OUT p_moved INT)
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
        BEGIN ROLLBACK; RESIGNAL; END;
    START TRANSACTION;
    -- 1. approved ideas join the corpus
    -- 2. their tags follow them
    -- 3. this semester closes and the next one opens
    -- 4. an audit row records the move
    COMMIT;
END
```

A.4 Overlap.of: weighted overlap of two term sets (Java)

```
public static double of(Set<String> mine, Set<String> theirs,
                       TermStats stats) {
    Set<String> shared = new HashSet<>(mine);
    shared.retainAll(theirs);
    if (shared.isEmpty()) return 0.0;
    Set<String> union = new HashSet<>(mine);
    union.addAll(theirs);
    double s = stats.weight(shared);
    return 0.8 * (s / stats.weight(union))
        + 0.2 * (s / stats.weight(mine));
}
```

A.5 DbFolderWatcher.scan: read a file once it stops changing (Java, abridged)

```
public synchronized int scan() throws IOException {
    for (Path file : filesIn(folder)) {
        String version = size(file) + ":" + modified(file);
        // read only when unchanged since the previous scan
        boolean stable = version.equals(lastSeen.put(name, version));
        if (stable && !log.seen(name, size, modified)) {
            service.ingest(file); // one transaction per file
        }
    }
}
```

Annex B: User Manual

Origin needs JDK 17 or newer, Apache Maven 3.9 and MySQL 8 running on port 3306.

- Step 1: Clone the repository: `git clone [repository URL]`
- Step 2: Install dependencies: Maven downloads them on the first build (mvn compile).
- Step 3: Configure the settings: copy `config/origin.properties.example` to `config/origin.properties` and enter the MySQL password.
- Step 4: Set up the database: nothing to run by hand. On first start Origin creates the origin database, the schema, the triggers, the procedures and the demo data. To add real past projects, copy CSV, TXT, XLSX or DOCX files into the DB folder next to `run.bat`; Origin reads them within about ten seconds.
- Step 5: Start the application: double-click `run.bat`, or run `mvn package` and deploy `target/origin.war` to Tomcat 10.1.
- Step 6: Access the application at: `http://localhost:8080`

Demo accounts use the password Origin@2026: students sandesh@origin.edu, bhumika@origin.edu and diwakar@origin.edu (group G09), students of groups G01 to G08 (for example aarati@origin.edu), and supervisor@origin.edu. To start again with fresh demo data, drop the origin database and start Origin once more.

A typical session: open the home page and type an idea after "I want to build"; the map shows how much of that ground is taken. Sign in, press Run the full check, complete the form and read the result after the evaluation screen. Edit and check again if the idea overlaps, then lock it for the group. The supervisor drops past-project files into the DB folder, reviews the locked ideas on the supervisor desk, and closes the semester when every group is approved.